

ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

MÁSTER UNIVERSITARIO EN INGENIERÍA DEL SOFTWARE E
INTELIGENCIA ARTIFICIAL

**Mejora del contexto en LLMs para la generación de
pruebas unitarias**

Enhancing context in LLMs for unit test generation

Realizado por

Alfonso Cabezas Fernández

Tutorizado por

Dr. Gabriel Jesús Luque Polo

Dr. Francisco Javier Ferrer Urbano

Departamento

Lenguajes y Ciencias de la Computación

UNIVERSIDAD DE MÁLAGA

MÁLAGA, Julio 2026



E.T.S. INGENIERÍA
INFORMÁTICA

UNIVERSIDAD DE MÁLAGA



UNIVERSIDAD DE MÁLAGA

TRABAJO DE FIN DE MÁSTER

Máster en Ingeniería del Software e Inteligencia Artificial

Mejora del contexto en LLMs para la generación de pruebas unitarias

Manual técnico y manual de código, despliegue y uso

Autor: Alfonso Cabezas Fernández

Directores: Dr. Gabriel Jesús Luque Polo

Dr. Francisco Javier Ferrer Urbano

12 de julio de 2026

*Dedicado a
mi familia*

Agradecimientos

Quisiera aprovechar este espacio para mostrar mi agradecimiento a mi familia por toda su ayuda y apoyo.

En primer lugar me gustaría agradecer a mis padres Juan Antonio y Nieves por su esfuerzo, cariño y apoyo constante en mi vida y en especial, durante el transcurso de mi vida académica y el desarrollo de este proyecto. Su presencia ha sido fundamental en cada etapa de mi vida y mi educación.

Del mismo modo me gustaría agradecer a mi hermana Ana y a mi tío Paco, que también han sido un gran apoyo en mi vida, siempre dispuestos a ayudar cuando lo necesitaba. Gracias por estar ahí.

Finalmente, agradezco a mis abuelas por su cariño y por interesarse siempre por mi progreso y bienestar.

A todos vosotros, gracias por vuestro apoyo.

Resumen

Este trabajo de fin de máster presenta una propuesta innovadora para mejorar el contexto que reciben los Modelos de Lenguaje de Gran Tamaño (LLMs) durante la generación y regeneración automática de pruebas unitarias. La solución se basa en la integración de técnicas de Generación Aumentada por Recuperación (RAG) para enriquecer el contexto del modelo, utilizando embeddings generados con el modelo microsoft/graphcodebert-base [1], un índice vectorial construido con FAISS [2] y la incorporación de información relevante extraída del archivo pom.xml. Además, se utiliza la API Mistral con Codestral 25.01 [3] para generar pruebas unitarias de alta calidad en proyectos de software. La metodología propuesta ha sido evaluada en el proyecto JInstagram [4], considerando diversos porcentajes de contexto (100 %, 60 %, 40 % y 10 %) para analizar su impacto en la cobertura y calidad de las pruebas generadas.

Los resultados indican que el uso de un contexto completo mejora significativamente la cobertura de sentencias y ramas, superando a enfoques existentes como ChatUniTest [5] o incluso los casos de prueba creados por los mismos desarrolladores del proyecto. Este estudio contribuye a la reducción del esfuerzo manual en la escritura de pruebas y abre nuevas vías para la integración de LLMs en el ciclo de desarrollo de software.

Palabras clave: Pruebas Unitarias, LLMs, RAG, Contexto



Abstract

This master's thesis presents an innovative approach to enhancing the context provided to Large Language Models (LLMs) during the automatic generation and regeneration of unit tests. The solution is based on integrating Retrieval Augmented Generation (RAG) techniques to enrich the model's context by using embeddings generated with the model microsoft/graphcodebert-base [1], a vector index built with FAISS [2], and incorporating relevant information extracted from the pom.xml file. Additionally, the Mistral API with Codestral 25.01 [3] is employed to generate high-quality unit tests for software projects. The proposed methodology has been evaluated on the JInstagram project [4], using various context percentages (100 %, 60 %, 40 %, and 10 %) to analyze its impact on the coverage and quality of the generated tests.

The results indicate that using complete context significantly improves statement and branch coverage, outperforming existing approaches such as ChatUniTest [5] and even the tests created by the project's own developers. This study contributes to reducing the manual effort required for writing tests and opens up new avenues for integrating LLMs into the software development lifecycle.

Keywords: Unit Testing, LLMs, RAG, Context



Índice de Contenidos

Índice de Contenidos	XIII
Índice de Figuras	XV
Índice de Tablas	XVII
1. Introducción	1
1.1. Objetivos	3
1.2. Metodología	3
1.3. Fases de trabajo	4
1.4. Organización	4
2. Antecedentes	7
2.1. Historia y evolución de la generación de pruebas unitarias	7
2.2. Uso de modelos de lenguaje en generación de pruebas	9
2.3. Comparación de herramientas existentes	11
2.3.1. EvoSuite	11
2.3.2. Randoop	11
2.3.3. AthenaTest y A3Test	12
2.3.4. ChatUniTest	13
2.3.5. TestART	15
2.3.6. Otras herramientas híbridas	16
2.4. Uso de RAG para mejorar el contexto en generación de pruebas . . .	17
3. Descripción del problema	21
3.1. Limitación del contexto en LLMs	21
3.1.1. Impacto en la comprensión de la implementación	22
3.1.2. Consecuencias en la generación de pruebas unitarias	22

3.2. Relevancia del problema	23
4. Detalles de la propuesta	25
4.1. Introducción a la propuesta general	25
4.1.1. Flujo de ejecución	26
4.2. Decisiones y tecnologías empleadas	27
4.2.1. Generación Aumentada por Recuperación (RAG)	27
4.2.1.1. Uso del RAG en esta propuesta	29
4.2.2. Embeddings y representación del código	30
4.2.2.1. Elección de <code>microsoft/graphcodebert-base</code>	32
4.2.3. Índice vectorial FAISS: funcionamiento y justificación	33
4.2.4. Distancia L2 para búsqueda de similitud	34
4.2.5. API del LLM: Codestral 25.01	36
4.3. Diseño y uso de los <i>prompts</i>	38
4.3.1. Descripción de los <i>prompts</i> empleados	38
4.3.2. Justificación del diseño de los <i>prompts</i>	41
4.3.3. Inclusión de información del archivo <code>pom.xml</code>	42
4.3.3.1. Importancia del <code>pom.xml</code> para generar pruebas	42
4.3.3.2. Cómo se integra esta información en el <i>prompt</i>	43
4.3.3.3. Ventajas de este método	43
4.4. Compilación automática del código	44
4.4.1. Importancia de compilar cada clase	44
4.4.2. Uso de <code>javac</code> para detectar errores	45
4.4.3. Justificación del uso de 3 intentos de regeneración	45
4.4.4. Integración en el flujo de generación	46
4.5. Registro de estadísticas y generación de <i>logs</i>	46
4.5.1. Registro de datos en el archivo CSV	47
4.5.2. Generación de <i>logs</i> en <code>.txt</code> y análisis de resultados	48
4.5.3. Justificación y utilidad del registro de estadísticas	49
4.6. Selección del proyecto de pruebas: JInstagram	50
4.6.1. Motivos para elegir JInstagram	50
4.7. Propuesta de experimentación	51
4.7.1. Motivación y enfoque experimental	52
4.7.2. Metodología de evaluación	52

5. Experimentación y resultados	55
5.1. Resumen y análisis de resultados	55
5.1.1. Comparación con ChatUniTest	55
5.1.2. Comparación de cobertura según contexto	57
5.1.3. Resumen detallado de las métricas analizadas	58
5.1.4. Análisis de la ejecución de los tests según el porcentaje de contexto	60
5.1.5. Resumen de tiempos de generación de tests	61
5.1.6. Resumen de tiempos totales del proceso	62
5.2. Discusión final del análisis	63
6. Conclusiones y trabajo futuro	65
6.1. Conclusiones	65
6.2. Trabajo futuro	66
Anexo A. Manual de código, despliegue y uso	69
A.1. Código fuente y organización del proyecto	69
A.2. Prerrequisitos	69
A.2.1. Dependencias Python	70
A.2.2. Archivo <code>requirements.txt</code>	71
A.2.3. Creación y configuración del archivo <code>.env</code>	71
A.3. Configuración del <code>pom.xml</code> para Maven	72
A.4. Despliegue y uso	74
A.4.1. Ejecución del script de generación de pruebas	74
Bibliografía	80



Índice de figuras

2.1. Ciclo de generación de ChatUniTest [5]	14
4.1. Funcionamiento básico de un LLM con RAG	29



Índice de tablas

5.1. Comparación de resultados de pruebas: humanas, ChatUniTest [7] y la propuesta (con 100 % de contexto) para JInstagram [4]	56
5.2. Cobertura de pruebas según el porcentaje de contexto recuperado . .	57
5.3. Resumen del informe del CSV y consumo de tokens	58
5.4. Resultados de ejecución de tests según porcentaje de contexto	60
5.5. Resumen de tiempos de generación de tests por porcentaje de contexto	61
5.6. Resumen de tiempos totales del proceso por porcentaje de contexto .	62



1. Introducción

Las pruebas unitarias son una parte fundamental en el desarrollo de software, ya que permiten detectar errores y garantizar la fiabilidad del código antes de su integración y puesta en funcionamiento en diferentes sistemas [6]. Sin embargo, la escritura manual de pruebas unitarias es un proceso laborioso y costoso en términos de tiempo y recursos [7]. Para mitigar estos desafíos, se han desarrollado técnicas de generación automática de pruebas, entre las cuales los Modelos de Lenguaje de Gran Tamaño (o LLMs, por sus siglas en inglés) han mostrado un gran potencial [8].

Los LLMs, como ChatGPT, DeepSeek o CodeLlama, han sido entrenados con grandes cantidades de código, lo que les permite generar pruebas unitarias de manera automática. No obstante, estos modelos presentan limitaciones significativas, especialmente en términos de cobertura, corrección y, por lo tanto, calidad de las pruebas generadas.

Investigaciones recientes han demostrado que una proporción significativa de los tests generados con estas herramientas contienen errores de compilación, fallos de ejecución y cobertura insuficiente, lo que afecta su fiabilidad y capacidad de aplicación en la práctica [9]. Un factor determinante en estos problemas es que el contexto con el que trabajan los modelos está limitado a la cantidad máxima de información que estos pueden procesar, lo que les impide usar el proyecto entero como contexto cuando este es demasiado grande, dificultando, por tanto, la comprensión de la implementación específica de ciertas áreas de este.

Para abordar esta limitación, se han explorado diversas estrategias para mejorar la generación de pruebas unitarias mediante LLMs. Una de ellas es la Generación Aumentada por Recuperación (RAG, por sus siglas en inglés), la cual permite a los LLMs acceder a información externa en tiempo real para aumentar su contexto y mejorar la generación de pruebas [8]. Estas técnicas pueden mejorar la cobertura del código utilizando documentación básica, documentación de APIs, issues de GitHub o respuestas de StackOverflow como fuentes adicionales de información. Sin embargo, la eficacia de los RAGs en la generación de pruebas unitarias sigue siendo un área en exploración [8].

Además de los RAGs, existen otras estrategias para mejorar el contexto en la generación de pruebas con LLMs. Herramientas como ChatUniTest [5] o TestART [10] han sido desarrolladas para proporcionar a los LLMs información relevante del contexto extraída directamente del propio código. ChatUniTest usa tanto técnicas de análisis de código y recuperación de información para seleccionar contexto como técnicas de reparación para mejorar la respuesta del LLM y, por ende, los tests generados por este.

Recientemente, en otra investigación se ha propuesto TestART, una herramienta que también mejora la generación de pruebas unitarias, en este caso mediante la coevolución de generación y reparación automatizada, logrando mejoras en la tasa de éxito y la cobertura en comparación con los modelos base [10]. Estas herramientas permiten reducir errores de compilación y mejorar la cobertura al proporcionar un contexto más completo a la hora de generar los tests [5, 10].

Este estudio se centra en la mejora del contexto en LLMs para la generación de pruebas unitarias. La investigación evaluará también cómo la inclusión de diferentes cantidades de información relevante en forma de contexto en el proceso de generación afecta a la cobertura de los tests.

Al proporcionar un enfoque más estructurado para la recuperación de información sobre el contexto, este estudio busca contribuir a la evolución de las técnicas

CAPÍTULO 1. INTRODUCCIÓN

de generación automática de pruebas unitarias y su aplicabilidad en entornos de desarrollo reales.

1.1. Objetivos

El objetivo principal de este Trabajo de Fin de Máster es realizar un estudio para mejorar el contexto disponible en los Modelos de Lenguaje de Gran Tamaño (LLMs) para la generación de pruebas unitarias.

Se tratará de enriquecer el contexto utilizado por los LLMs a la hora de generar pruebas unitarias, con el fin de mejorar la cobertura y reducir los errores de compilación de los tests generados, incrementando, por lo tanto, la fiabilidad de las pruebas generadas.

Con los resultados obtenidos, se evaluará cómo la inclusión de diferentes contextos en el proceso de generación afecta a la cobertura de los tests generados.

1.2. Metodología

Este estudio seguirá una metodología basada en la investigación, la experimentación y el análisis de resultados. Se utilizará un enfoque empírico para mejorar el contexto disponible y evaluar cómo la variación de este afecta a la generación de pruebas unitarias mediante LLMs con la herramienta de generación de contexto escogida.

Para ello, tras determinar y configurar la herramienta, se realizarán experimentos controlados en los que se compararán distintos contextos y los resultados que producen en el LLM a la hora de generar las pruebas unitarias. Se aplicarán métricas como la cobertura del código o el número de errores de compilación y ejecución para determinar la calidad de las pruebas generadas.

Para realizar las mediciones mencionadas anteriormente, se emplearán herramien-

tas de análisis de código o frameworks de pruebas para medir la eficacia de cada enfoque.

Finalmente, se analizarán los datos obtenidos y se extraerán conclusiones sobre la relación entre el contexto y la cobertura de las pruebas generadas.

1.3. Fases de trabajo

Las fases del trabajo serán las siguientes:

1. **Revisión bibliográfica:** Se recopilará y analizará literatura relevante sobre generación de pruebas unitarias con LLMs y técnicas de mejora de contexto.
2. **Definición del entorno experimental:** Se seleccionará la herramienta que se usará para generar el contexto, el modelo de lenguaje y el proyecto a utilizar en la experimentación.
3. **Implementación de los experimentos:** Se configurarán los modelos, la herramienta de generación de contexto y se probarán distintos tipos de este en la generación de pruebas unitarias.
4. **Evaluación de los resultados:** Se analizarán métricas como la cobertura y los errores de compilación para determinar la fiabilidad de los tests generados.
5. **Documentación y conclusiones:** Se elaborará la memoria del trabajo, resumiendo los hallazgos, exponiendo las conclusiones del estudio y proponiendo posibles mejoras o futuras líneas de investigación.

1.4. Organización

El trabajo se estructurará en seis capítulos, cada uno de ellos tratará aspectos fundamentales que permitan comprender en detalle tanto el contexto teórico como

CAPÍTULO 1. INTRODUCCIÓN

la propuesta desarrollada. A continuación, se describe brevemente cada uno de los capítulos:

- **Capítulo 1: Introducción:** Se presentará el contexto del estudio, se exponen los objetivos, la metodología y las fases de trabajo.
- **Capítulo 2: Antecedentes:** Se realizará una revisión del estado del arte, abarcando la evolución de la generación automática de pruebas unitarias y el uso de modelos de lenguaje. Además, se compararán diversas herramientas existentes.
- **Capítulo 3: Descripción del Problema:** Se detallarán las limitaciones de contexto inherentes a los Modelos de Lenguaje de Gran Tamaño y se analizará cómo estas limitaciones afectan a la calidad y la cobertura de las pruebas unitarias generadas.
- **Capítulo 4: Detalles de la propuesta:** Se expondrá la solución propuesta para mejorar el contexto en la generación de pruebas.
- **Capítulo 5: Experimentación y resultados:** Se presentarán y analizarán los resultados de los experimentos realizados para evaluar la eficacia de la propuesta.
- **Capítulo 6: Conclusiones y Trabajo Futuro:** Se resumirán los principales hallazgos, se extraerán conclusiones relevantes y se plantearán posibles líneas de mejora y futuras investigaciones que permitan seguir avanzando en el campo de la generación automática de pruebas unitarias.

Esta estructura permite una comprensión clara del trabajo, comenzando con los fundamentos teóricos, para posteriormente centrarnos en la propuesta técnica, la experimentación y el análisis de los resultados obtenidos. De este modo, obtenemos una presentación coherente y lógica del proceso de investigación y desarrollo llevado a cabo.



2. Antecedentes

En este capítulo se describirán ciertos aspectos clave para entender el desarrollo realizado en este trabajo fin de máster, así como un estudio del estado del arte y una comparativa que muestra las aportaciones y relevancia de las contribuciones realizadas con otras soluciones ya existentes.

2.1. Historia y evolución de la generación de pruebas unitarias

La generación automática de pruebas unitarias ha sido objeto de investigación durante décadas. Los enfoques tradicionales, que se basan principalmente en técnicas de análisis de programas y análisis de cobertura, surgieron para intentar maximizar la detección de fallos reduciendo el trabajo manual. Herramientas como EvoSuite [11] o Randoop [12] representan hitos en esta evolución. Randoop implementa pruebas aleatorias guiadas por retroalimentación (o *feedback-directed random testing*), generando secuencias de llamadas aleatorias a métodos y utilizando la ejecución de dichas secuencias para evitar entradas inválidas o repetidas.

Por otra parte, EvoSuite utiliza algoritmos evolutivos para construir suites de pruebas que puedan alcanzar niveles altos de cobertura estructural (por ejemplo, cobertura de líneas y de ramas). EvoSuite trata la generación de casos de prueba como un problema de optimización guiado por una función de aptitud o *fitness*

CAPÍTULO 2. ANTECEDENTES

function orientada a la cobertura, la cual ha demostrado su eficacia a la hora de conseguir coberturas elevadas en diversos contextos. Sin embargo, estos métodos tradicionales generan casos de prueba muy diferentes a los escritos manualmente por desarrolladores.

Fraser y Arcuri señalan en [11] que, aunque EvoSuite puede generar asertos para verificar ciertos comportamientos, las pruebas generadas suelen ser difíciles de leer y entender por humanos. De hecho, herramientas como EvoSuite y Randoop no tienen comprensión semántica del programa, lo cual deriva en pruebas limitadas o irrelevantes (por ejemplo, pruebas triviales sobre el estado del objeto) [11, 12].

La evolución de estos enfoques llevó a los investigadores a explorar nuevas técnicas basadas en aprendizaje automático. A medida que crecían los conjuntos de datos de código y las pruebas disponibles, surgieron modelos de lenguaje especializados en generación de código de prueba.

Un ejemplo relevante de estas nuevas técnicas es AthenaTest [13], que trata la generación de pruebas como un problema de traducción: con este enfoque se entrena un modelo basado en Transformers para traducir un método de código (o *focal method*) en un método de prueba. AthenaTest se entrenó con métodos y casos de prueba reales y logró generar pruebas más legibles y cercanas a las generadas por desarrolladores humanos. De hecho, en una comparación de legibilidad y utilidad, los desarrolladores que vieron las pruebas mostraron preferencia por las de AthenaTest frente a las de EvoSuite. No obstante, su precisión inicial era bastante limitada, ya que menos de un quinto de las pruebas generadas por AthenaTest fueron completamente correctas sin intervención humana [13].

Trabajos posteriores mejoraron este enfoque. En concreto, con A3Test [14] se añadió conocimiento específico de *assertions* y una verificación de *method signatures* o firmas de métodos de prueba para mejorar la cantidad de pruebas correctas y la cobertura. En una evaluación realizada posteriormente sobre miles de métodos, A3Test obtuvo un 147 % más de casos de prueba correctos y un 15 % más de métodos

cubiertos en comparación con AthenaTest. Además generó pruebas de forma más eficiente, lo que destaca la importancia y el valor de enriquecer el contexto y las verificaciones dentro del proceso de generación de las pruebas unitarias.

2.2. Uso de modelos de lenguaje en generación de pruebas

Con la aparición de los LLMs o Modelos de Lenguaje de Gran Tamaño aparecieron nuevas posibilidades para la generación automática de pruebas unitarias.

A diferencia de los modelos específicos entrenados exclusivamente para esta tarea, los LLMs generales (como por ejemplo, GPT-4.5, DeepSeekR1, etc.) poseen un gran conocimiento adquirido de grandes cantidades de código y texto, lo que les ha permitido tener una gran capacidad para comprender código y producir casos de prueba válidos sin entrenamiento adicional [15].

Estudios recientes han mostrado que LLMs sin *fine-tuning* específico pueden generar pruebas con niveles de cobertura notables si se les dan *prompts* bien diseñados. Por ejemplo, Schäfer et al. (2023) en [15] desarrollaron la herramienta TestPilot para JavaScript, la cual usa GPT-3.5 turbo para generar pruebas unitarias a partir de la cabecera e implementación de una función dada, complementadas con ejemplos de uso extraídos de la documentación de la API.

TestPilot logró una mediana de 70.2% de cobertura de sentencias y 52.8% de cobertura de ramas en un conjunto de 25 paquetes de *npm*, superando significativamente técnicas tradicionales de prueba para JavaScript. Además, se le añadió un mecanismo de retroalimentación por el cual, si una prueba generada falla, el sistema reintroduce en el *prompt* el mensaje de error y la prueba fallida para que el LLM intente corregirla; esta estrategia que actualmente se conoce como “regeneración” mejoró la calidad y corrección de los casos de prueba resultantes [15].

Los LLMs aportan grandes ventajas en esta tarea. Primero, tienden a generar casos de prueba que en general, como se ha probado anteriormente, son más entendibles para los humanos y con *assertions* más significativas que las generadas con enfoques basados únicamente en la cobertura.

Esto se debe a que pueden aprovechar el conocimiento de patrones que hay en el código y prácticas comunes en los tests que han aprendido al entrenarse con millones de ejemplos. Por ejemplo, las pruebas producidas por AthenaTest se consideraron más legibles y útiles que las de las herramientas tradicionales basadas en búsqueda [13].

Además, los LLMs pueden adaptarse dinámicamente al contexto proporcionado en el *prompt*, ya que si se les dan los detalles del código para el que realizamos el test, pueden determinar condiciones lógicas y configuraciones de objetos necesarios para ejecutar el método.

Sin embargo, el uso de LLMs en la generación de pruebas también tiene limitaciones y desventajas. Una de las más destacadas es que son propensos a la “alucinación” (el modelo puede inventar llamadas a métodos, clases o suposiciones incorrectas sobre el código), resultando en casos de prueba que no compilan o que generan excepciones al ejecutarse [15].

Sin mecanismos eficaces de verificación, un gran porcentaje de las pruebas generadas automáticamente pueden ser erróneas o incompletas; por ejemplo, antes de aplicar técnicas de reparación, muchas pruebas generadas por ChatGPT presentan fallos de compilación [15].

Otro problema es que, aunque los LLMs tienen conocimiento general de dominios de programación, no tienen información específica o contexto del proyecto o librerías propias que se usan en el mismo, especialmente si ese contexto excede el tamaño de contexto manejable por el modelo.

Esto limita enormemente su capacidad para generar pruebas con proyectos muy grandes o muy específicos.

2.3. Comparación de herramientas existentes

En la literatura reciente podemos ver bastantes herramientas de generación automática de pruebas, cada una con enfoques y resultados diferentes. A continuación, se comparan algunas de las más importantes, desde el punto de vista de sus principales ventajas y desventajas. Dichas herramientas son:

2.3.1. EvoSuite

EvoSuite [11] representa una herramienta de los métodos tradicionales basados en búsqueda.

Una de sus principales ventajas es que puede generar automáticamente conjuntos de pruebas con una alta cobertura estructural (como líneas y ramas de código) sin necesidad de especificaciones adicionales. Para lograrlo, usa heurísticas evolutivas que le permiten explorar caminos de ejecución complejos. Además, puede generar *assertions* simples que capturan el comportamiento actual del programa, lo que facilita la detección de futuras regresiones.

Sin embargo, también tiene desventajas. Los casos de prueba generados suelen ser difíciles de leer para los desarrolladores. Debido a su falta de comprensión semántica, EvoSuite tiende a incluir *assertions* triviales, como verificar que un objeto no sea `null` justo después de crearlo. Esto puede dificultar el mantenimiento de las pruebas y también reducir su utilidad.

2.3.2. Randoop

Randoop [12] es una herramienta que genera pruebas automáticamente mediante técnicas aleatorias con retroalimentación.

Entre sus principales ventajas está la capacidad de descubrir errores inesperados gracias a la exploración pseudoaleatoria. También pueden generar una gran cantidad

CAPÍTULO 2. ANTECEDENTES

de casos de prueba en poco tiempo, incluso sin conocimiento previo sobre el código. Ha demostrado eficacia detectando excepciones no manejadas y errores de contrato en bibliotecas Java simplemente combinando llamadas de forma aleatoria.

Por otra parte, tiene algunas desventajas importantes. Al no centrarse en objetivos específicos de cobertura (solo intenta evitar la redundancia), puede generar muchas pruebas irrelevantes o repetitivas. Además, sus *assertions* suelen limitarse únicamente a verificar que no ocurran errores evidentes, sin validar estados específicos ni resultados esperados en detalle, lo que limita la utilidad de las pruebas generadas.

2.3.3. AthenaTest y A3Test

AthenaTest [13] y A3Test [14] forman parte de la primera generación de herramientas que utilizan aprendizaje profundo para generar casos de prueba automáticamente.

Su principal ventaja es que producen pruebas similares a las que escribiría un desarrollador humano, tanto en estilo como en estructura. Esto se debe a que aprenden a partir de grandes conjuntos de código real. Por ejemplo, AthenaTest usa un modelo basado en Transformer entrenado con cientos de miles de ejemplos de métodos y sus pruebas reales, generando pruebas con llamadas a bibliotecas y nombres de variables que se parecen a los que escribimos las personas [13]. Las pruebas generadas por AthenaTest destacan por su claridad y legibilidad frente a las que generan herramientas tradicionales basadas en búsqueda. A3Test mejora todavía más estos resultados incorporando buenas prácticas en las *assertions* (por ejemplo, aprendiendo cuándo usar correctamente condiciones de tipo `assert`) y mejorando la coherencia entre el nombre de la prueba y el método. Gracias a esto, A3Test genera pruebas con menos errores lógicos, aumentando por tanto la validez de las pruebas generadas automáticamente y la cobertura en comparación con AthenaTest.

No obstante, estas herramientas también presentan desventajas. Una de ellas es la dificultad para asegurar que las pruebas generadas sean totalmente correctas. Por

CAPÍTULO 2. ANTECEDENTES

ejemplo, AthenaTest inicialmente generaba muchas pruebas que no podían ejecutarse sin modificaciones debido a errores en *assertions* o configuraciones incompletas, problema que A3Test consiguió reducir parcialmente [13, 14]. Incluso con A3Test, la cobertura alcanzada en algunos casos puede ser inferior a herramientas centradas en la cobertura, como EvoSuite, especialmente cuando la lógica del código es distinta a los patrones que han aprendido durante el entrenamiento. Por último, entrenar y ajustar estos modelos puede ser costoso, y su rendimiento depende mucho de la cantidad y calidad de los datos disponibles.

2.3.4. ChatUniTest

ChatUniTest [5] es una herramienta relativamente reciente y en constante evolución que utiliza un modelo de lenguaje (concretamente ChatGPT) integrado en un proceso de generación, validación y reparación de pruebas.

Entre sus principales ventajas está que utiliza un mecanismo llamado *contexto focal adaptativo*, el cual extrae automáticamente información importante del proyecto (por ejemplo, el método que se quiere probar y sus dependencias cercanas, como métodos auxiliares o clases relacionadas) para incluirla en el *prompt* que se envía al modelo. Gracias a esto, el modelo dispone de un contexto muy específico sobre el código analizado, lo que le permite generar casos de prueba más relevantes. Además, ChatUniTest usa un proceso iterativo de generación/reparación que podemos ver en más detalle en la Figura 2.1: primero obtiene la respuesta del LLM, luego verifica si el código generado compila y se ejecuta correctamente, y si se detectan errores aplica un módulo de reparación. Este módulo corrige errores sencillos usando reglas (por ejemplo, importando bibliotecas faltantes o corrigiendo errores de sintaxis), mientras que, para errores más complicados, usa nuevamente el LLM con información sobre el problema detectado. Este proceso reduce significativamente los errores típicos de compilación que suelen tener las respuestas directas generadas por un LLM. En

CAPÍTULO 2. ANTECEDENTES

diversas evaluaciones, ChatUniTest alcanzó la cobertura más alta de líneas y ramas comparado con herramientas anteriores, superando incluso a EvoSuite en más de la mitad de los casos estudiados. También consiguió una cobertura superior en métodos específicos comparado con AthenaTest y A3Test [5].

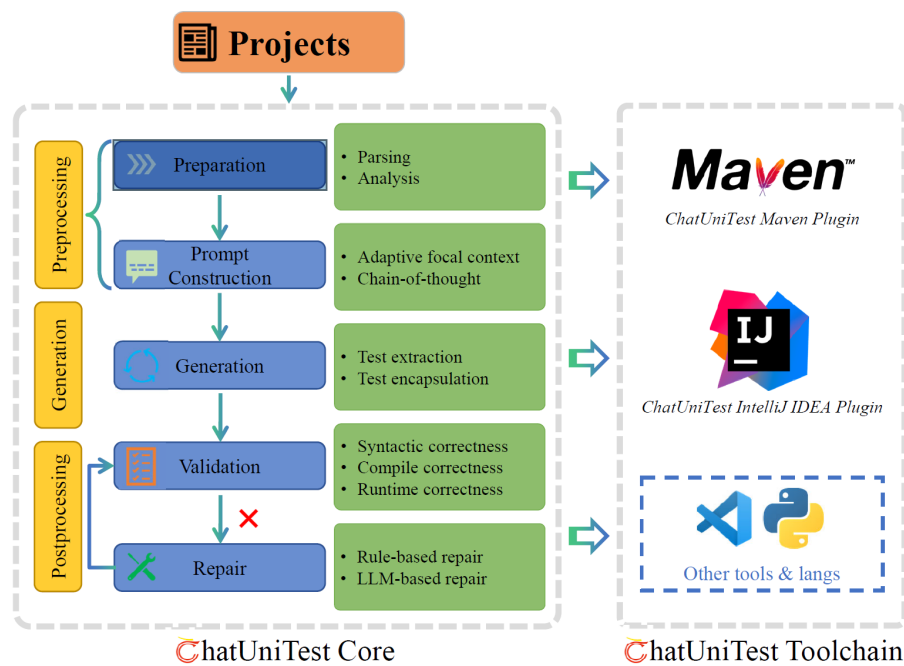


Figura 2.1: Ciclo de generación de ChatUniTest [5]

Sin embargo, también tiene algunas desventajas. Al depender de un LLM externo como ChatGPT, pueden darse problemas sobre los costes computacionales o de uso del LLM y la confidencialidad del código. La calidad de las pruebas generadas también puede reducirse cuando el código con el que se está trabajando es muy extenso o complicado, ya que resumir suficiente contexto dentro de los límites del modelo puede ser difícil. Además, el proceso iterativo de validación y reparación puede aumentar el tiempo requerido en comparación con otras herramientas más directas.

2.3.5. TestART

TestART [10] también es una herramienta reciente que combina generación automática con modelos de lenguaje y reparación guiada por cobertura.

Una de las principales ventajas de TestART es que utiliza ChatGPT-3.5 para generar inicialmente los casos de prueba y, luego, aplica una estrategia de reparación basada en plantillas para corregir errores comunes en las pruebas generadas, como fallos de compilación o ejecución. Estas plantillas permiten resolver de una forma rápida problemas frecuentes, como ajustar tipos incorrectos, manejar excepciones no capturadas o añadir *assertions* que faltan, sin la necesidad de usar el modelo de lenguaje constantemente, lo que acelera el corregir errores. Además, TestART extrae información de cobertura de las pruebas correctas para identificar qué partes del código no están siendo probadas y usa esta información para crear nuevas pruebas con el LLM en otras iteraciones. Gracias a esto, cada iteración mejora la cobertura obtenida anteriormente. También incorpora técnicas para tratar de evitar respuestas repetitivas por parte del modelo, mediante modificaciones en el *prompt*. En pruebas realizadas, TestART ha conseguido aumentar la cantidad de pruebas válidas y mejorar la cobertura respecto a otros enfoques que no utilizan ciclos de reparación iterativos. Además, alcanzó una cobertura superior a EvoSuite generando solo la mitad de casos de prueba, lo que demuestra que sus pruebas son más eficientes y completas [10].

Entre sus desventajas está la complejidad de todo el proceso, lo que lleva a tener un coste computacional mucho mayor y posibles dificultades a la hora de poder integrarse en flujos de desarrollo continuos, ya que necesitan resultados inmediatos. Aunque TestART reduce considerablemente las alucinaciones, ya que tiene reparación automática, sigue dependiendo en gran medida de la calidad de las respuestas del modelo. Esto hace que puedan hacer falta ciclos adicionales o ajustes manuales, especialmente en casos con lógica muy compleja o en dominios muy específicos, donde el LLM puede no entender bien la funcionalidad del código.

2.3.6. Otras herramientas híbridas

Existen otras herramientas híbridas que también merecen mencionarse. Por ejemplo, TestPilot [15] combina la generación automática mediante modelos de lenguaje con ejemplos extraídos de la documentación del código. Además, utiliza un ciclo en el que vuelve a intentar la generación en caso de fallos, demostrando que mejora considerablemente el rendimiento frente a herramientas tradicionales, especialmente con código JavaScript.

Por otro lado, CODAMOSA [16] integra modelos de lenguaje (por ejemplo, OpenAI Codex) con técnicas de búsqueda evolutiva. Cuando la búsqueda tradicional llega a un límite en la cobertura (también conocido como “plateau”), CODAMOSA utiliza el modelo para generar casos de prueba adicionales que cubran áreas no exploradas del código. Estos casos generados por LLM se incorporan después al conjunto ya existente para seguir con el proceso de búsqueda evolutiva. Este método híbrido ha mostrado resultados prometedores al superar estancamientos de cobertura, mejorando la eficiencia en la generación automática de pruebas en comparación con técnicas basadas solo en búsqueda evolutiva.

En resumen, las herramientas que usan modelos de lenguaje están teniendo un gran crecimiento y alcanzando o incluso superando a métodos tradicionales en métricas como la cobertura y los desarrolladores las consideran cada vez más útiles. Esto ocurre sobre todo cuando incluyen mecanismos diseñados para solventar las principales debilidades que tienen los LLMs (como las alucinaciones o el contexto limitado), usando estrategias de validación y reparación.

2.4. Uso de RAG para mejorar el contexto en generación de pruebas

Una tendencia reciente en este campo es usar técnicas basadas en RAG (o Generación Aumentada por Recuperación) para dar un mejor contexto a los modelos de lenguaje. Básicamente, este enfoque consiste en buscar o extraer información útil del código o del entorno antes de llamar al modelo. Luego, esta información se incluye en el *prompt* para guiar o mejorar la respuesta del modelo. El objetivo principal es complementar la falta de conocimiento específico del modelo y superar las limitaciones del contexto que este puede manejar, ofreciéndole datos concretos sobre el proyecto que no podría conocer por sí solo [5].

Este método, como se ha expuesto anteriormente, ha tenido éxito en diferentes desarrollos con LLMs, como en *chatbots* que consultan información de bases de datos o documentos. Además, trabajos previos han demostrado que la incorporación de fuentes externas mediante técnicas RAG no solo mejora la precisión de las respuestas de los modelos de lenguaje [17, 18], sino que también se ha extendido a tareas de ingeniería de software, como la generación de código [19] y la reparación automatizada de programas [20]. Estos estudios muestran el potencial de RAG para enriquecer el contexto en la generación de pruebas unitarias.

Por ejemplo, ChatUniTest [5] utiliza también la tecnología RAG de *contexto focal adaptativo*. Este le permite analizar primero el código estáticamente para identificar el método a probar y sus dependencias más cercanas (por ejemplo, otras funciones relacionadas o clases importadas). Luego, incluye estos fragmentos clave en el *prompt* enviado al modelo. De esta forma, el LLM no solo recibe el método para el cual se quieren generar los tests, sino también información relevante para entender su contexto dentro del proyecto.

Otro ejemplo es TestPilot [15], que añade al *prompt* fragmentos de la docu-

CAPÍTULO 2. ANTECEDENTES

mentación oficial de las APIs que muestran cómo deberían usarse correctamente las funciones para las que queremos generar pruebas. Al proporcionar al modelo ejemplos reales de uso, facilita la generación de casos de prueba relevantes y bien estructurados.

Por otro lado, RATester [21] usa otro enfoque: recupera de forma literal información clave del propio código fuente. Para cada método a probar en proyectos escritos en Go, RATester utiliza el servidor de lenguaje `gopls` para obtener definiciones de tipos, implementaciones relacionadas y comentarios del código, lo que evita que el modelo genere detalles incorrectos sobre funciones o variables desconocidas.

También existen estrategias que combinan RAG con métodos de generación en diferentes fases. Por ejemplo, CasModaTest [22] propone una arquitectura en cascada, que es independiente del modelo usado, en la cual primero se recupera automáticamente contexto útil (como funciones o datos relacionados), y después este contexto seleccionado se envía al LLM para generar las pruebas. Este proceso iterativo se intenta que el modelo reciba solo información esencial y relevante, lo que mejora notablemente la calidad de las pruebas generadas, independientemente del modelo elegido.

En todos estos casos, el uso de RAG ha demostrado mejorar significativamente la calidad de las pruebas generadas automáticamente. Yin et al. [21], por ejemplo, concluyen que al usar su método de recuperación precisa de contexto, consiguieron aumentar considerablemente la cobertura del código comparado con otros enfoques sin recuperación adicional. Además, lograron detectar más fallos al aprovechar esta información del código. Del mismo modo, TestPilot demostró que incluir ejemplos concretos en el *prompt* es fundamental para mejorar la cobertura, ya que eliminarlos reducía el rendimiento [15]. En el caso de ChatUniTest, el uso de contexto adaptativo permitió generar asertos más efectivos e incluso utilizar técnicas avanzadas como objetos simulados (o *mocks*) y reflexión cuando era necesario, incrementando la cobertura en las partes complejas del código [5].

CAPÍTULO 2. ANTECEDENTES

En resumen, las implementaciones que usan RAG solucionan muchas de las limitaciones de los modelos de lenguaje, ya que les permiten tener información específica o contexto sobre el sistema que se está probando. Como resultado, las pruebas generadas contienen menos errores, menos alucinaciones y generan *assertions* más correctas. Todo esto hace que las pruebas automáticas sean cada vez más parecidas en calidad a las creadas manualmente por los desarrolladores [5, 10].



CAPÍTULO 2. ANTECEDENTES

3. Descripción del problema

La generación automática de pruebas unitarias mediante Modelos de Lenguaje de Gran Tamaño (o LLMs) tiene una limitación importante: estos modelos solo pueden manejar una cantidad limitada de información (o tokens) en cada llamada. Debido a esto, no pueden tener en cuenta el proyecto completo como contexto, lo que hace que les sea mucho más difícil entender a fondo cómo funciona específicamente cada parte del sistema.

Esta limitación influye enormemente en la calidad y utilidad de las pruebas generadas, especialmente en proyectos muy grandes o complejos. Además, impide aprovechar todo el potencial de estos modelos al integrarlos en desarrollos reales de gran escala.

En esta sección se describen estos problemas con más detalle y se explica por qué es importante mejorar y enriquecer el contexto proporcionado a los modelos, para poder obtener mejores resultados en la generación automática de pruebas unitarias.

3.1. Limitación del contexto en LLMs

Los modelos de lenguaje como ChatGPT, pueden procesar solo una cantidad limitada de tokens en cada ejecución o llamada. Debido a este límite, a la hora de elegir qué información se les va a dar, es necesario elegir partes específicas del proyecto y dejar fuera otra información importante que puede estar repartida en varios archivos o módulos. En proyectos grandes, esta reducción del contexto implica

perder detalles sobre la estructura del sistema, las dependencias entre módulos y la lógica interna del código que puede ser clave a la hora de generar las pruebas [6].

3.1.1. Impacto en la comprensión de la implementación

La limitación del contexto afecta directamente la capacidad del modelo para entender cómo se relacionan y cómo funcionan internamente algunas de las partes del sistema. Al trabajar con solo una pequeña parte del proyecto:

- El modelo pierde información sobre relaciones importantes entre paquetes, funciones, clases y módulos, dificultando que entienda el comportamiento general del sistema.
- No se tienen en cuenta detalles sobre configuraciones y/o dependencias críticas, lo que impide reconocer condiciones o el funcionamiento específico de algunas partes del proyecto que son necesarias para generar las pruebas correctamente.
- Al tener una representación incompleta del código, el modelo puede realizar interpretaciones erróneas o inventarse la información faltante, generando pruebas que pueden tener errores de compilación o de ejecución [9].

3.1.2. Consecuencias en la generación de pruebas unitarias

La limitación del contexto afecta directamente a la calidad de las pruebas unitarias generadas:

- **Cobertura incompleta:** Al no conocer el proyecto completo, el modelo no puede cubrir todos los escenarios relevantes, lo que reduce significativamente la eficacia de las pruebas generadas.
- **Generación de pruebas triviales:** Debido a la falta de información, el modelo suele crear pruebas básicas que solo comprueban condiciones simples o triviales, sin tratar realmente la lógica compleja del sistema.

- **Errores durante la ejecución:** La falta de detalles y relaciones importantes puede hacer que las pruebas generadas tengan problemas al compilar o que fallen durante su ejecución [7, 5].

3.2. Relevancia del problema

La limitación del contexto es un problema importante al generar automáticamente pruebas unitarias, ya que, principalmente:

- Impide que el modelo pueda evaluar correctamente todo el sistema.
- Provoca que las pruebas generadas no reflejen bien la complejidad ni las relaciones internas del código.
- Reduce la fiabilidad y utilidad de las pruebas cuando se aplican en situaciones reales de desarrollo.
- Reduce la cobertura de las pruebas generadas.
- Puede llevar a la generación de pruebas triviales o con errores tanto de compilación como de ejecución.

En resumen, al no poder considerar todo el proyecto como contexto, el modelo tiene dificultades para entender cómo está implementado el sistema. Esto lleva a pruebas unitarias incompletas y con errores frecuentes. Esta limitación es especialmente grave en proyectos grandes o complejos, donde comprender todo el sistema es fundamental para garantizar que se generan pruebas de calidad [6, 7, 9, 5].



CAPÍTULO 3. DESCRIPCIÓN DEL PROBLEMA

4. Detalles de la propuesta

En este capítulo se explica de forma detallada cómo funciona el sistema propuesto para mejorar el contexto que utilizan los Modelos de Lenguaje (LLMs) al generar automáticamente pruebas unitarias. La solución combina técnicas como la Generación Aumentada por Recuperación (RAG), embeddings generados con el modelo `microsoft/graphcodebert-base`, la creación de un índice vectorial con FAISS, el diseño específico de prompts, la inclusión de información obtenida del archivo `pom.xml`, el uso de la API Codestral (modelo `codestral-latest`, versión 25.01) y el registro de estadísticas durante las ejecuciones. También se explican y justifican las pruebas propuestas para evaluar la efectividad del contexto proporcionado y el proyecto seleccionado para realizar dichas pruebas.

A continuación, se presentan las secciones y subsecciones que explican estos aspectos.

4.1. Introducción a la propuesta general

El objetivo principal de esta propuesta es mejorar el contexto que reciben los Modelos de Lenguaje (LLMs) cuando generan pruebas unitarias automáticamente. Para conseguir esto, se utilizan técnicas que permiten extraer información relevante del código, documentación, archivos de configuración y otros elementos importantes del proyecto, lo que se conoce como Generación Aumentada por Recuperación (RAG). El objetivo es aumentar la calidad, cobertura y corrección de las pruebas generadas,

CAPÍTULO 4. DETALLES DE LA PROPUESTA

minimizando errores de compilación y ejecución.

La idea principal es que al proporcionar al modelo más información específica del proyecto (por ejemplo, fragmentos de código importantes o información obtenida del `pom.xml`), el modelo puede entender mejor el código y, por lo tanto, generar pruebas unitarias más útiles para los desarrolladores [23, 15].

4.1.1. Flujo de ejecución

A continuación, se detalla paso a paso cómo funciona el sistema implementado. El código desarrollado se encarga de realizar diversas tareas importantes que se explican a lo largo de este capítulo.

El sistema se compone de diversas funcionalidades que trabajan en conjunto para:

1. Leer y procesar los archivos con el código fuente del proyecto.
2. Generar embeddings (representaciones numéricas) de cada archivo usando el generador seleccionado.
3. Construir un índice vectorial con FAISS (una biblioteca que permite almacenar y buscar rápidamente los vectores que son más similares a partir de un embedding) para facilitar la búsqueda del contexto más relevante.
4. Extraer información útil del proyecto (por ejemplo, propiedades del archivo `pom.xml`) para mejorar el contexto del prompt.
5. Generar las clases de prueba para cada clase del proyecto. Para ello:
 - a) Consultar el índice FAISS y extraer los 10 índices más relevantes que formarán el contexto que se le suministrará al LLM.
 - b) Generar el prompt que se enviará a la API del modelo para la creación de pruebas unitarias.

CAPÍTULO 4. DETALLES DE LA PROPUESTA

- c) Enviar el prompt a la API y recibir la respuesta del modelo con el código generado.
- d) Compilar el código generado para asegurarnos de que funciona correctamente. Si el código tiene errores de compilación, se inicia un proceso de regeneración que intenta corregir estos errores hasta tres veces. Si después de tres intentos el código sigue sin compilar, se descarta.
- e) Registrar información detallada de cada ejecución (tokens usados, tiempo de respuesta, éxito o fracaso en la compilación, etc.) en archivos CSV y logs en formato .txt.

6. Ejecutar las pruebas generadas y crear informes sobre los resultados obtenidos.

Cada una de estas acciones funcionalidades se implementa en el código escrito en Python, utilizando tanto librerías estándar como otras específicas para procesamiento de lenguaje y manejo de vectores (por ejemplo, `transformers`, `faiss` y `torch`). Como se ha mencionado anteriormente, el proceso es iterativo: si una prueba generada no compila, el código y el error detectado se envían nuevamente al modelo para intentar corregir o regenerar la prueba hasta tres veces.

4.2. Decisiones y tecnologías empleadas

Antes de explicar paso a paso cómo funciona la solución propuesta, primero es importante aclarar cuáles son las tecnologías y herramientas elegidas, y las justificaciones de estas decisiones.

4.2.1. Generación Aumentada por Recuperación (RAG)

La Generación Aumentada por Recuperación (RAG) es una técnica que combina modelos generativos (como los LLM) con mecanismos que recuperan información

CAPÍTULO 4. DETALLES DE LA PROPUESTA

externa adicional. Dicho de otra forma, en lugar de que el modelo solo utilice el conocimiento aprendido durante su entrenamiento y almacenado en sus parámetros, el RAG permite buscar información relevante desde fuentes externas en tiempo real y luego incorporarla al contexto proporcionado al modelo. Ante una consulta, el sistema recupera fragmentos relevantes de una base de datos o índice vectorial y los proporciona como contexto adicional al modelo generativo. De este modo, el modelo puede aumentar su contexto con información específica recuperada en tiempo real.

Diversos estudios han demostrado que usar RAG mejora notablemente el rendimiento en tareas donde se requiere conocimiento específico. Por ejemplo, Lewis et al. (2020) [17] mostraron que los modelos que usan RAG generan respuestas más precisas y mejor fundamentadas en comparación con modelos que no usan recuperación externa.

Esto sucede porque el uso de RAG permite acceder a información actualizada y específica (por ejemplo, documentación de APIs o fragmentos de código similares existentes), lo cual ayuda al modelo a generar resultados más correctos. En el contexto de tareas de programación, el RAG es especialmente útil porque puede recuperar ejemplos reales de código o comentarios que ayuden al modelo en la generación de pruebas o soluciones más adecuadas.

Otros trabajos confirman el impacto positivo de incorporar RAG en programación. Por ejemplo, Parvez et al. (2021) [24] introdujeron el *framework REDCODER*, que recupera fragmentos relevantes de código y comentarios para ayudar al modelo a generar código o resúmenes. Este estudio mostró claras mejoras en tareas de generación de código Java y Python gracias al contexto adicional recuperado.

En resumen, como se observa en la Figura 4.1, el funcionamiento básico de RAG es el siguiente:

1. Se recibe una consulta o *prompt* inicial y se genera el *embedding* de la misma.
2. Se utiliza un motor de búsqueda o un índice vectorial (por ejemplo, FAISS)

CAPÍTULO 4. DETALLES DE LA PROPUESTA

para recuperar vectores similares en una base o índice de conocimientos.

3. La información de los vectores más relevantes recuperada se concatena o se integra en el *prompt*.
4. El LLM procesa el *prompt* enriquecido y genera la respuesta deseada.

Al combinar recuperación de información con modelos de lenguaje, se obtienen respuestas más detalladas, precisas y útiles. En la generación automática de pruebas unitarias, esto significa poder cubrir casos específicos o ejemplos basados en implementaciones reales del proyecto.

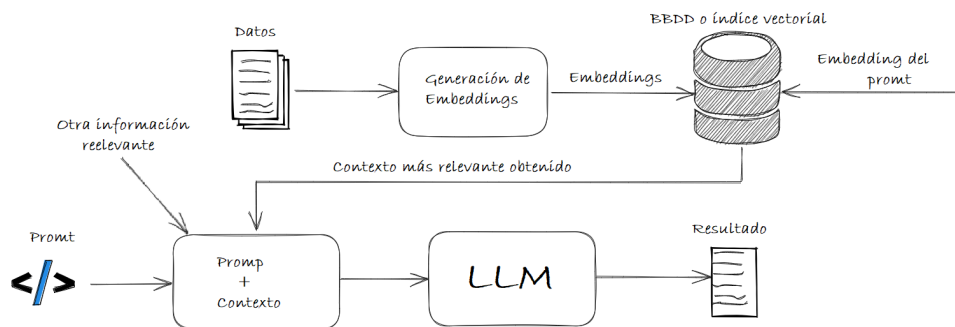


Figura 4.1: Funcionamiento básico de un LLM con RAG

4.2.1.1. Uso del RAG en esta propuesta

En la propuesta se implementa un RAG personalizado mediante las siguientes etapas:

1. **Generación de embeddings:** Se generan embeddings para las diferentes clases del código del proyecto y se almacenan en un índice FAISS.
2. **Extracción del contexto:** Usando el índice FAISS y distancia L2, se recupera contexto relevante para la clase que se quiere probar.

CAPÍTULO 4. DETALLES DE LA PROPUESTA

3. **Enriquecimiento del prompt:** Esta información recuperada, junto con otros datos relevantes que se explicarán después, se añade al prompt que se envía al modelo. Esto mejora el contexto y, por tanto, la calidad de las pruebas generadas por el modelo.
4. **Iteración en la generación:** Si una prueba generada no compila, se incorpora el código generado y la información de los errores de compilación detectados y se vuelve a enviar al modelo para intentar regenerar esa clase de test hasta en tres ocasiones. Esta decisión se justificará más adelante.

Para entender en profundidad lo que sucede con los dos primeros pasos: Primero se coge el código original, se trocea en fragmentos más pequeños que puede procesar el generador embeddings y se generan. Esa representación es un vector, no el código en sí. Luego, estos vectores se indexan con FAISS para permitir búsquedas basadas en similitud. Cuando se hace una consulta, esta se vectoriza de forma similar y se compara con los vectores indexados para encontrar los más cercanos. Finalmente, se usan las referencias a estos vectores para recuperar los fragmentos referenciados del código original. Es decir, la vectorización sirve como un paso intermedio para facilitar la búsqueda de fragmentos por similitud, y el código que se recupera es el que está referenciado a esos vectores, no se traduce un vector en código, se coge el fragmento de código al que hace referencia el vector.

Este enfoque permite superar la limitación del contexto del modelo al proporcionarle información específica del proyecto que normalmente quedaría fuera debido a la restricción de tokens del mismo.

4.2.2. Embeddings y representación del código

Como se ha mencionado anteriormente, los embeddings son representaciones vectoriales que capturan las características semánticas de un dato, como una palabra o, en nuestro caso, un fragmento de código.

CAPÍTULO 4. DETALLES DE LA PROPUESTA

Un embedding es un vector formado por diferentes valores continuos entrenado de tal forma que los elementos con significados similares quedan cercanos en el espacio vectorial. Este concepto apareció en trabajos como *word2vec* de Mikolov et al. (2013) [25], que demostró que es posible crear automáticamente representaciones vectoriales donde relaciones semánticas y sintácticas (como por ejemplo, rey - hombre $\approx$ reina - mujer) se conservan mediante operaciones algebraicas.

Cuando se generan embeddings a partir del código, estos permiten representar funciones, clases o fragmentos de código de tal manera que partes con comportamientos similares estén cerca en ese espacio vectorial. Por ejemplo, dos funciones con lógica parecida (aunque usen diferentes nombres de variables o métodos) deberían tener vectores similares. Esto no es sencillo debido a la complejidad del código, pero estudios recientes han avanzado bastante en este problema. Por ejemplo, Alon et al. (2019) [26] presentaron un modelo llamado *code2vec*, demostrando que estas representaciones capturan la semántica real del código.

Otro ejemplo relevante es CodeBERT [27], un modelo entrenado con grandes cantidades de código comentado utilizando Transformers. CodeBERT genera embeddings capaces de relacionar descripciones en lenguaje natural con implementaciones en código fuente, obteniendo excelentes resultados en tareas como búsqueda y generación automática de documentación.

La importancia principal de los embeddings radica en que convierten partes del código (palabras clave, estructuras, funciones) en puntos en un espacio vectorial. Esto permite medir la similitud semántica entre ellos (usando distancias como la L2 o similitud por coseno). De este modo, es posible implementar búsquedas semánticas rápidas y precisas, como ocurre con RAG: al realizar una consulta (por ejemplo, contexto de una determinada clase para la que queremos generar una prueba), obtenemos su embedding y buscamos código similar existente en el proyecto. Esto aporta al modelo información muy importante para generar mejores pruebas.

CAPÍTULO 4. DETALLES DE LA PROPUESTA

4.2.2.1. Elección de `microsoft/graphcodebert-base`

El modelo `GraphCodeBERT-base` de Microsoft es una versión mejorada de `CodeBERT` que incluye información sobre la estructura del código, lo cual lo hace una muy buena opción para nuestra propuesta. `GraphCodeBERT` fue presentado por Guo et al. (2021) [1] para resolver la limitación de modelos anteriores (como `CodeBERT`), que solo veían el código como secuencias lineales de texto, ignorando información estructural importante.

`GraphCodeBERT` añade durante el entrenamiento la información del flujo de datos (o *data flow*) del código mediante un grafo que conecta variables con sus usos. Esto permite al modelo aprender dependencias y flujos de información entre partes del código. Gracias a esta información adicional, `GraphCodeBERT` genera embeddings más detallados, obteniendo mejores resultados en tareas como búsqueda de código, detección de código duplicado y refinamiento automático de código. Por ejemplo, en la búsqueda de código, `GraphCodeBERT` no solo se basa en la proximidad del texto, sino que también entiende conexiones más profundas entre variables y estructuras.

En comparación con `CodeBERT`, `GraphCodeBERT` obtiene mejores resultados precisamente porque aprovecha esta estructura interna del código, algo especialmente relevante al generar pruebas unitarias. Además, frente a modelos más recientes como `CodeT5` o `Codex`, `GraphCodeBERT` ofrece la ventaja de ser abierto, sencillo de usar e ideal para tareas basadas en embeddings (sin necesidad de que sea generativo), además de ser fácil de integrar en la etapa de recuperación.

En resumen, el modelo `microsoft/graphcodebert-base` se ha elegido porque es capaz de capturar tanto la semántica como la estructura profunda del código. Diferentes estudios recientes han confirmado que `GraphCodeBERT` es especialmente eficaz para análisis de código, gracias a su entrenamiento con grandes cantidades de código real y su arquitectura basada en Transformers, que permite comprender mejor las relaciones internas del código [1].

4.2.3. Índice vectorial FAISS: funcionamiento y justificación

Para realizar búsquedas rápidas y eficientes dentro de grandes cantidades de embeddings de código, se ha escogido FAISS (*Facebook AI Similarity Search*), una biblioteca optimizada para buscar rápidamente los vectores más parecidos a uno dado, especialmente en espacios vectoriales con muchas dimensiones [28].

El funcionamiento básico de FAISS consiste en construir estructuras de datos especializadas que permiten, dado un vector de consulta, encontrar rápidamente vectores cercanos (similares) dentro del índice FAISS. Para medir esta similitud, utilizamos la distancia L2 (distancia euclídea), la cual será explicada más adelante.

FAISS permite tanto búsquedas exactas (por ejemplo, comparando exhaustivamente todos los vectores, usando la GPU para realizar más rápido las operaciones) como búsquedas aproximadas más rápidas usando técnicas como la *cuantización* o la división del espacio vectorial en grupos (o *clusters*). Por ejemplo, el índice IVF (Inverted File Index) organiza los vectores en clústers y solo explora los más relevantes durante la búsqueda, acelerando mucho la búsqueda [28].

Además, FAISS está muy optimizado ya que está implementado en C++ (con soporte para Python) aprovechando arquitecturas hardware tipo SIMD y GPUs, permitiendo manejar bases de datos gigantes manteniendo tiempos de respuesta muy bajos. Johnson et al. (2017) [28] mostraron cómo FAISS podía manejar búsquedas en bases con mil millones de vectores, siendo hasta 8.5 veces más rápido que otros métodos disponibles.

Estas propiedades son muy importantes en nuestro caso, ya que nuestro sistema basado en RAG necesita realizar búsquedas rápidas sobre muchos embeddings de funciones o fragmentos de código cada vez que se genera una prueba.

Aunque existen otras opciones para realizar búsquedas vectoriales, como Annoy (Spotify) [29] o HNSW [30] (que usa grafos navegables), FAISS destaca por su alto rendimiento. También existen bases de datos vectoriales completas (por ejemplo,

CAPÍTULO 4. DETALLES DE LA PROPUESTA

Milvus, Pinecone o Weaviate) que usan internamente FAISS o HNSW, pero en nuestro caso, preferimos usar FAISS directamente para tener más control sobre su configuración.

Para la propuesta, se ha optado por recuperar los 10 índices más relevantes a la hora de generar el contexto, ya que esto permite obtener un equilibrio óptimo entre cantidad y calidad de la información del contexto. Con esta elección se garantiza que el modelo tenga acceso a fragmentos de código que, en conjunto, aporten una visión lo suficientemente amplia y precisa del contexto relevante, sin sobrecargar el prompt con información innecesaria. Esto favorece una generación de tests de calidad, al centrarse en los datos que realmente aportan valor.

En resumen, FAISS es una solución rápida, escalable y muy personalizable para realizar búsquedas eficientes en grandes bases de embeddings. Gracias a su rendimiento, incluso con grandes cantidades de vectores, podemos recuperar rápidamente los fragmentos de código más relevantes por orden de relevancia para mejorar el contexto proporcionado al modelo en tiempo real.

4.2.4. Distancia L2 para búsqueda de similitud

La distancia L2, también conocida como distancia euclídea, es la métrica que se ha escogido para comparar embeddings y determinar cuáles son más similares. Matemáticamente, la distancia L2 entre dos vectores $\mathbf{u}$ y $\mathbf{v}$ se define así:

$$\|\mathbf{u} - \mathbf{v}\|_2 = \sqrt{\sum_i (u_i - v_i)^2}.$$

En la práctica, muchos índices vectoriales (como FAISS) utilizan la distancia L2 al cuadrado (sin calcular la raíz cuadrada), ya que esto simplifica cálculos y no cambia el orden de cercanía entre vectores (ya que es monótonamente creciente con la L2 y evita el coste computacional de la raíz, sin cambiar el orden de cercanía).

Hemos escogido la distancia L2 por varias razones:

CAPÍTULO 4. DETALLES DE LA PROPUESTA

1. Es una métrica estándar y ampliamente utilizada. Herramientas como FAISS la utilizan por defecto en muchos de sus índices (por ejemplo, `IndexFlatL2` que calcula distancias euclídeas directamente) y está altamente optimizada en hardware.
2. Cuando los embeddings no están normalizados, la distancia euclídea considera tanto la dirección como la magnitud del vector. En cambio, la similitud coseno solo mide el ángulo (dirección), ignorando la longitud.

Para embeddings normalizados, la distancia L2 y la similitud coseno son equivalentes. De hecho, la similitud coseno se define como:

$$\text{coseno}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}$$

y se comporta de forma similar a L2 cuando los vectores están normalizados.

Aunque podríamos usar similitud coseno con vectores normalizados, la distancia L2 es una opción efectiva y más simple para nuestra propuesta. Además, la elección de métrica puede depender de la dimensión y distribución de los datos, ya que diversos estudios han demostrado que, en espacios de muy alta dimensión, las diferencias entre distintas métricas de distancia crecen (se conoce como “concentración de la distancia”) [31]. Sin embargo, dado que nuestros embeddings tienen dimensiones moderadas, la distancia L2 es una opción buena y efectiva.

De hecho, muchas técnicas de *hashing* y proyección aleatoria para búsqueda de vecinos cercanos asumen L2; por ejemplo, el esquema de *Locality-Sensitive Hashing* con distribuciones p -estables fue desarrollado para esta métrica [32].

En conclusión, se ha elegido la distancia L2 porque es una métrica sencilla, eficiente, compatible con FAISS y muy buena para medir la similitud entre vectores que representan fragmentos de código, ofreciendo rapidez y precisión al recuperar vectores similares.

4.2.5. API del LLM: Codestral 25.01

Codestral 25.01 [3] es un modelo avanzado de lenguaje desarrollado específicamente para generar código, creado por Mistral AI en enero de 2025.

Su disponibilidad mediante API (tanto desde la plataforma de Mistral como integrado en Azure AI Foundry de Microsoft) facilita su uso en nuestro sistema, ya que permite enviar solicitudes de generación de código sin tener que mantener la infraestructura propia de un modelo enorme.

Internamente, Codestral es un modelo Transformer muy grande (aunque la cifra exacta de parámetros que tiene no se ha publicado, ha superado a modelos de la categoría sub-100B, dentro de la que se enmarca), entrenado especialmente para entender y escribir código en más de 80 lenguajes de programación, incluyendo lenguajes como Python, Java, C++ y JavaScript, y también otros menos comunes o antiguos como Go, Ruby y Fortran. Esto lo convierte en una opción muy buena para diferentes tipos de proyectos.

Una característica destacable de Codestral es su capacidad de manejar hasta 256,000 tokens de contexto, muy superior a la mayoría de los modelos actuales (que suelen manejar entre 4,000 y 32,000 tokens). Esta capacidad tan amplia permite procesar archivos grandes o múltiples archivos juntos, y permite tener un contexto más completo durante la generación de código.

En nuestro caso (la generación de pruebas unitarias), esto significa que podemos incluir no solo la función que queremos probar, sino también otros fragmentos relevantes del código sin preocuparnos por quedarnos sin espacio en el contexto.

Además, Codestral ha sido entrenado especialmente para completar código en medio de archivos (llamado *fill-in-the-middle*, o FIM), lo cual, aunque esta funcionalidad no sea útil para nuestra propuesta (ya que tratamos de generar clases de test completas) sí lo es para generar pruebas unitarias dentro de archivos ya existentes. Esto indica que Codestral es especialmente competente para entender

CAPÍTULO 4. DETALLES DE LA PROPUESTA

contexto parcial.

Otro beneficio importante es su velocidad: Codestral 25.01 genera código aproximadamente el doble de rápido que su versión anterior, Codestral 24.05 según se indica en [3]. De hecho, es más rápido que modelos comparables como CodeLlama-70B, que además tienen una ventana de contexto más limitada (4,000 tokens). Esta rapidez lo hace ideal para situaciones en las que se necesitan respuestas rápidas o donde hay una gran cantidad de solicitudes.

Desde el punto de vista de la integración, Codestral ofrece una gran flexibilidad ya que puede usarse como servicio en la nube (en nuestro caso a través de la API proporcionada por Mistral), o también puede ser instalado directamente en infraestructuras privadas si la privacidad es un requisito importante.

Al comparar con otras opciones:

- **Respecto a otros modelos *open-source*:** Aunque modelos abiertos como StarCoder (15B) o CodeLlama (34B/70B) también son gratuitos, ninguno tiene el mismo nivel de contexto de Codestral (256,000 tokens) ni su especialización en FIM. Por ejemplo, aunque CodeLlama-34B tiene versiones que manejan hasta 100,000 tokens, su rendimiento en tareas de generación de código sigue siendo inferior al que proporciona Codestral 25.01 [3].
- **Respecto a modelos de pago (por ejemplo, OpenAI GPT-4):** Aunque GPT-4 es un modelo potente, no está exclusivamente optimizado para generación de código. Codestral tiene ventajas en velocidad, rendimiento especializado y, por supuesto, un menor coste (en nuestro caso, usamos la versión de prueba que es gratuita).

Además, Codestral ofrece a los desarrolladores un control más detallado, ya que se puede determinar hasta dónde se expande el contexto de código, o afinar el comportamiento con *prompts* especializados.

CAPÍTULO 4. DETALLES DE LA PROPUESTA

En resumen, Codestral 25.01 es una solución potente y optimizada para generar pruebas unitarias, gracias a su amplio contexto, gran rendimiento en la generación de código y velocidad superior a otros modelos disponibles. Estas características hacen que encaje perfectamente en esta propuesta basada en RAG, ya que puede utilizar eficientemente el contexto enriquecido que se le dé para generar pruebas unitarias completas, precisas y coherentes. Además, mediante la API podemos obtener datos útiles como el número de tokens usados y el tiempo de respuesta, información que registramos para mejorar continuamente el sistema [3].

4.3. Diseño y uso de los *prompts*

La calidad de las pruebas unitarias generadas por un modelo de lenguaje depende en gran parte del diseño del *prompt* que se utiliza. En esta propuesta, se han diseñado *prompts* enriquecidos con información extraída directamente del proyecto, como el código fuente de la clase para la que se quieren generar los tests, detalles obtenidos del archivo `pom.xml` y contexto recuperado mediante el RAG. Esta sección explica cómo están diseñados estos *prompts*, qué información contienen, y cómo se utilizan durante el proceso de generación y corrección de las pruebas.

4.3.1. Descripción de los *prompts* empleados

El *prompt* se crea combinando varias fuentes de información para darle al modelo un contexto lo más completo y claro posible. El proceso sigue los siguientes pasos:

1. Se incluye el contenido completo de la clase que se quiere probar.
2. Se añade información importante extraída del archivo `pom.xml`, por ejemplo, configuraciones importantes como la versión de JUnit utilizada.
3. Se incorpora el contexto generado por el RAG, utilizando un índice FAISS

CAPÍTULO 4. DETALLES DE LA PROPUESTA

basado con embeddings generados con el modelo `graphcodebert-base` de Microsoft.

4. Como se muestra en el Listado 4.1, se establecen instrucciones claras para el modelo, especificando que debe generar y cómo.

A continuación en el Listado 4.1, se muestra un fragmento del código real utilizado para construir estos *prompts* en el sistema:

```
1 # Incluir propiedades del pom.xml
2 prompt_pom = f"Ten en cuenta que en el pom.xml hay las siguientes
   propiedades:\n{POM_PROPERTIES}\n\n"
3
4 # Construir el prompt base
5 base_prompt = (
6     "Genera el código completo de las pruebas unitarias únicamente
   de esta clase Java de un proyecto Maven con JUnit.
7     El resultado debe ser un archivo Java 100% compilable, sin
   comentarios, sin delimitadores markdown, y sin texto extra
   con la clase con los tests. Además no puedes inventar nada,
   ni añadir nuevas dependencias o imports. "
8     "La clase es:\n" + class_content + "\n\n" + prompt_pom +
9     "Para ayudarte a construir los tests ten en cuenta este contexto
   sobre el proyecto o sus imports:\n" + context + "\n\n"
10 )
```

Listado 4.1: Construcción del prompt base

En este código se concatena el contenido de la clase para la que queremos generar los tests (almacenado en `class_content`), la información extraída del `pom.xml` (variable `POM_PROPERTIES`) y el contexto obtenido a partir de la recuperación mediante FAISS (almacenado en la variable `context`). Se impone la restricción de que el código

CAPÍTULO 4. DETALLES DE LA PROPUESTA

generado debe ser 100 % compilable y no puede contener comentarios adicionales ni delimitadores markdown.

Cuando la primera generación falla (el código no compila), se utiliza el *prompt* modificado que podemos ver en el Listado 4.2 para la regeneración. En este caso se añade el test generado anteriormente y los errores de compilación que se han detectado, para que el LLM pueda corregir dichos errores.

```
1 extra_info = (  
2     "Este es el test que has generado antes (que contiene errores)  
3     \n" + prev_test_code +  
4     "Los errores de compilación que contiene son: \n" + prev_error +  
5     "\n\n"  
6 )  
7 prompt_to_send = (  
8     "Dada la siguiente clase Java de un proyecto Maven con JUnit:\n"  
9     + class_content + "\n\n" + extra_info +  
10    "Corrige dichos errores y genera el código completo de las  
    pruebas unitarias para esta clase. Es MUY IMPORTANTE que me  
    des solamente el código corregido sin comentarios  
    adicionales." + prompt_pom +  
11    "El resultado debe ser un archivo Java 100% compilable, sin  
    comentarios, sin delimitadores markdown, y sin texto extra  
    con la clase con los tests. Además no puedes inventar nada,  
    ni añadir nuevas dependencias o imports. "  
12    "Para ayudarte a construir los tests ten en cuenta este contexto  
    sobre el proyecto o sus imports:\n" + context + "\n\n"  
13 )
```

Listado 4.2: Construcción del prompt para regeneración

En este segundo caso, se incorpora la variable `prev_test_code` (el test generado

CAPÍTULO 4. DETALLES DE LA PROPUESTA

previamente) y `prev_error` (el error de compilación obtenido). Así, el *prompt* de regeneración le muestra al modelo los fallos que han ocurrido, permitiendo que el LLM genere mejor las correcciones necesarias. Esta estrategia de regeneración iterativa se basa en trabajos como [15], que muestra cómo la inclusión de la retroalimentación de error en el *prompt* mejora significativamente el número de pruebas compilables generadas.

4.3.2. Justificación del diseño de los *prompts*

El diseño de los *prompts* se basa en diferentes estudios que demuestran la importancia de suministrar un contexto rico y bien organizado para mejorar la calidad de las respuestas de los modelos de lenguaje. En concreto, destacan tres aspectos fundamentales:

1. **Contexto enriquecido:** Al incluir fragmentos relevantes del código y datos extraídos del archivo `pom.xml`, el modelo recibe información específica del proyecto. Lewis et al. (2020) [17] demostraron que añadir esta información mejora considerablemente la precisión y relevancia de las respuestas en tareas donde se necesita conocimiento específico. Del mismo modo, Schäfer et al. (2023) [15] descubrieron que proporcionar un contexto adecuado reduce las *alucinaciones* del modelo y genera mejores respuestas.
2. **Estructura y claridad:** Los *prompts* están diseñados de forma clara, directa y sencilla, evitando peticiones complejas o innecesarias. Esto facilita que el modelo entienda exactamente qué se le está pidiendo y genere respuestas más precisas. Feng et al. (2020) [27] recomiendan esta forma tan directa de generar los *prompts* porque mejora la calidad de la respuesta generada.
3. **Retroalimentación iterativa:** Se incluye información sobre errores de compilación en los *prompts* de regeneración. Estudios recientes demuestran que dar

CAPÍTULO 4. DETALLES DE LA PROPUESTA

al modelo información específica sobre errores detectados le ayuda a corregirlos en los demás intentos, aumentando así la probabilidad de generar código que compila [1]. Este mecanismo es muy importante en nuestro sistema porque necesitamos que las pruebas generadas sean compilables.

En resumen, las decisiones sobre cómo generar los *prompts* que se usan en la propuesta están basadas en investigaciones recientes y buenas prácticas en generación de código mediante LLMs, las cuales indican claramente que un *prompt* bien diseñado es fundamental para obtener resultados de calidad [17, 15, 1].

4.3.3. Inclusión de información del archivo `pom.xml`

Añadir información del archivo `pom.xml` al contexto que se proporciona al modelo es fundamental para generar pruebas unitarias que realmente se ajusten a la configuración específica del proyecto.

El archivo `pom.xml` se define, por ejemplo, qué versión de JUnit y otras dependencias importantes se usan en el proyecto Maven. Estudios recientes destacan que incluir esta información en los *prompts* mejora la calidad de las pruebas generadas, ya que el modelo necesita conocer exactamente las librerías y versiones que se están utilizando [17, 15].

Si esta información se omite, el modelo podría generar pruebas que usen métodos o clases incompatibles, provocando errores de compilación o fallos durante la ejecución.

Para solucionar esto, se ha desarrollado una función en Python que analiza el archivo `pom.xml` y extrae automáticamente las propiedades relevantes que luego se añaden al *prompt*.

4.3.3.1. Importancia del `pom.xml` para generar pruebas

En el `pom.xml` se definen todas las dependencias y configuraciones necesarias en un proyecto Maven. Para la generación automática de pruebas unitarias, es especialmente

CAPÍTULO 4. DETALLES DE LA PROPUESTA

importante porque determina qué librerías de pruebas están disponibles (por ejemplo, JUnit o TestNG) y cuáles son sus versiones.

La versión de las dependencias es especialmente importante. Por ejemplo, las anotaciones usadas en los métodos de prueba pueden variar según la versión específica de JUnit. Proporcionar esta información al modelo reduce la posibilidad de generar pruebas incompatibles. Esto se verifica con lo que dicen Schäfer et al. (2023) [15], quienes destacan que un contexto actualizado sobre dependencias reduce considerablemente los errores en la generación automática de código.

4.3.3.2. Cómo se integra esta información en el *prompt*

El proceso para incluir la información del archivo `pom.xml` en el *prompt* es el siguiente:

1. Se lee el archivo `pom.xml` y se extraen las etiquetas y valores importantes.
2. Se crea un texto con esta información que indica al modelo sobre las dependencias y versiones específicas que debe utilizar al generar pruebas.
3. Al crear el *prompt* (junto con el código de la clase a probar y el contexto obtenido mediante RAG), se incluye esta información.
4. El modelo recibe el *prompt* completo y genera pruebas teniendo en cuenta exactamente la configuración del `pom.xml`.

4.3.3.3. Ventajas de este método

Incluir la información del archivo `pom.xml` en el *prompt* tiene varias ventajas claras:

- **Mayor precisión en las dependencias:** El modelo sabrá exactamente qué versión de JUnit u otras librerías debe utilizar, evitando errores de compilación por anotaciones o métodos incorrectos [17].

- **Consistencia:** Las pruebas generadas estarán alineadas con la configuración real del proyecto, y se evitarán conflictos con otras dependencias [27].
- **Menos alucinaciones:** Al contar con información precisa sobre las dependencias, el modelo evita inventar versiones o dependencias [15].

En resumen, añadir la información del `pom.xml` mejora considerablemente la calidad y precisión de las pruebas generadas. Esto reduce errores de compilación y ejecución. Esto se basa en recomendaciones recientes sobre la importancia de proporcionar al modelo un contexto completo y preciso durante la generación automática de código [1, 15].

4.4. Compilación automática del código

La compilación automática del código generado es una parte fundamental del proceso para generar las pruebas. En proyectos Maven, es necesario que todas las clases de prueba del proyecto compilen correctamente antes de poder ejecutarlas y obtener métricas de cobertura con JaCoCo [15]. Con que una clase de prueba no compile, el *build* del proyecto completo podría fallar, haciendo que no se puedan ejecutar las pruebas y obtener los informes de cobertura.

4.4.1. Importancia de compilar cada clase

En un flujo normal, Maven ejecuta la fase de *test* solo después de que la fase de *compile* haya terminado correctamente. Por eso, es fundamental verificar que todas las pruebas compilen antes de intentar ejecutar los tests. Si existen pruebas que no compilan, Maven no podrá avanzar a la siguiente fase, lo que impediría evaluar la cobertura y validar las pruebas generadas automáticamente [17].

Dado que la cobertura es uno de los principales indicadores de calidad de las pruebas unitarias (y un objetivo clave en la propuesta), asegurar que todas las clases

CAPÍTULO 4. DETALLES DE LA PROPUESTA

de prueba sean válidas y compilables es esencial. Cualquier fallo de compilación interrumpe el proceso de construcción y, por lo tanto, impide el análisis de cobertura.

4.4.2. Uso de `javac` para detectar errores

La estrategia propuesta para compilar automáticamente las pruebas generadas se basa en utilizar el compilador de Java (`javac`) individualmente para cada archivo generado. Este método tiene dos ventajas importantes:

1. **Identificación precisa del error:** Al compilar cada archivo por separado a medida que se va generando, es sencillo identificar qué prueba específica está fallando y la traza completa con el error de compilación. Esto es más fácil que hacerlo en bloque cuando se compilan los test con Maven, donde los mensajes de error pueden resultar confusos y mucho menos específicos [27].
2. **Detalle de los errores:** El compilador proporciona mensajes detallados (por ejemplo, indicando qué *import* falta). Estos errores se almacenan y pueden utilizarse como retroalimentación para que el modelo corrija o regenere la prueba en intentos posteriores [1].

Una vez capturados estos errores, el sistema puede intentar corregir automáticamente la prueba, reenviando los detalles del error al LLM para que haga las modificaciones necesarias. Si no es posible corregir una prueba después de 3 intentos, esta se elimina para asegurarnos de que el proyecto pueda construirse sin problemas.

4.4.3. Justificación del uso de 3 intentos de regeneración

Se ha optado por limitar la regeneración a tres intentos para tener un equilibrio entre la calidad del resultado, el tiempo y los costes de generación, evitando ciclos interminables de correcciones que podrían aumentar significativamente el tiempo total de procesamiento sin garantizar mejoras sustanciales. La experimentación muestra

CAPÍTULO 4. DETALLES DE LA PROPUESTA

que la tasa de reparación con tres intentos ya es muy baja, por lo que no interesa invertir más tiempo ni utilizar tokens adicionales que resulten en un uso innecesario de recursos. Tres intentos permiten identificar y corregir el error rápidamente si este se debe a errores puntuales en la generación.

4.4.4. Integración en el flujo de generación

La compilación se realiza automáticamente dentro del flujo de generación de pruebas. Cuando el modelo genera un archivo de prueba, este se guarda en la ubicación correcta del proyecto. Luego se ejecutan los comandos para compilar el código y las pruebas generadas. Si la compilación falla, el sistema captura el error y lo envía de nuevo al modelo, iniciando un nuevo ciclo de corrección automática.

Después de asegurarnos de que todas las pruebas generadas compilan correctamente, se ejecutan las pruebas para medir la cobertura del código.

En resumen, la compilación automática del código generado es fundamental para asegurar que las pruebas generadas sean válidas y que el análisis de cobertura pueda realizarse correctamente. Utilizar `javac` proporciona información útil para mejorar continuamente las pruebas generadas y evitar introducir errores en el proyecto.

4.5. Registro de estadísticas y generación de *logs*

Registrar en detalle lo que ocurre durante la ejecución del sistema es muy importante para evaluar cómo de eficiente es y cómo mejorar la generación de pruebas. Por eso, se ha implementado un sistema de registro que registra información relevante durante cada llamada al modelo y durante todo el proceso en general. Esta información se guarda en dos tipos de archivos: un archivo CSV y un *log* en formato de texto (`.txt`).

4.5.1. Registro de datos en el archivo CSV

Cada vez que se hace una llamada al LLM, se guardan varios datos importantes en un archivo CSV. Esto permite analizar fácilmente el rendimiento y la calidad de la generación de pruebas. Los campos registrados son:

- **id:** Un número único (incremental) que identifica cada llamada realizada al modelo.
- **clase:** Nombre y ubicación relativa de la clase para la cual se genera la prueba. Esto evita errores de análisis cuando hay clases con nombres iguales en diferentes módulos.
- **intento:** Indica cuántos intentos se han realizado para generar una prueba que compile correctamente (1, 2 o 3).
- **compila:** Indica si la prueba generada compila correctamente (1) o no (0). Este campo es clave para analizar la calidad y compilabilidad de las pruebas generadas.
- **caracteres:** Cantidad de caracteres del mensaje (*prompt*) enviado al modelo, para evaluar la longitud del contexto proporcionado.
- **prompt_tokens:** Cantidad de *tokens* del *prompt* que ha consumido el LLM, según informa la API del modelo.
- **completion_tokens:** Cantidad de *tokens* devueltos por el modelo en la respuesta.
- **total_tokens:** Suma de los tokens del *prompt* y de la respuesta, lo que mide el coste total de cada solicitud.
- **tiempo:** Tiempo que tarda el modelo en generar la respuesta desde que el LLM recibe la solicitud, esencial para evaluar su eficiencia.

CAPÍTULO 4. DETALLES DE LA PROPUESTA

Este registro permite analizar individualmente cada llamada al modelo y también evaluar globalmente el rendimiento del sistema. Por ejemplo, al revisar la cantidad de *tokens* y el tiempo de respuesta, se puede determinar si un *prompt* demasiado largo está afectando negativamente la rapidez. También se puede evaluar qué porcentaje de pruebas generadas compila correctamente desde el primer intento.

4.5.2. Generación de *logs* en *.txt* y análisis de resultados

Además del archivo CSV, se genera un *log* en formato de texto (*.txt*) que incluye de forma clara y ordenada:

- Toda la salida generada durante el proceso (errores, advertencias, mensajes del sistema).
- Tiempos de ejecución de cada fase, como la creación del índice FAISS, generación de pruebas, ejecución de pruebas con Maven y creación de informes, etc.

Usando ambos registros, se genera un informe final que resume los siguientes resultados:

- Número total de llamadas al modelo y clases del proyecto consideradas (tanto del código existente, como de pruebas generadas).
- Cantidad y porcentaje de pruebas que compilaban en el primer, segundo o tercer intento, así como las que no compilaban nunca.
- Promedio de caracteres y *tokens* medios utilizados por clase y globalmente.
- Promedio de *tokens* generados por el modelo en sus respuestas.
- Consumo total de *tokens* en *prompts* y respuestas.
- Tiempo promedio empleado para generar pruebas (por clase y globalmente) y tiempo total por fase del proceso.

CAPÍTULO 4. DETALLES DE LA PROPUESTA

El informe también detalla tiempos específicos dedicados a cada fase del sistema:

- Creación del índice FAISS y contexto (RAG).
- Generación de pruebas.
- Ejecución de pruebas con Maven.
- Generación del informe final.
- Tiempo total de ejecución del sistema.

Estos registros son fundamentales para entender cómo funciona el sistema, detectar posibles cuellos de botella y analizar y mejorar continuamente la eficiencia del proceso.

4.5.3. Justificación y utilidad del registro de estadísticas

Registrar estadísticas detalladas y generar *logs* es clave para evaluar, diagnosticar y optimizar el sistema de generación automática de pruebas. Estos datos permiten:

1. **Evaluar la eficiencia:** Al medir el uso de *tokens* y tiempos, se puede determinar rápidamente dónde ocurren retrasos o excesos en el consumo de recursos.
2. **Evaluar la calidad de generación:** Al monitorear la tasa de éxito (compilación correcta) y cantidad de intentos necesarios, se pueden identificar problemas en el diseño de los *prompts* o en la estrategia de generación.
3. **Mejorar iterativamente:** La información registrada sirve como base para ajustes futuros en el tamaño del contexto o en cómo se manejan los errores, algo recomendado por estudios recientes sobre generación automática de código [17, 15].
4. **Detectar errores rápidamente:** Registrar mensajes detallados de errores de compilación permite proporcionar al modelo retroalimentación específica para corregir las pruebas generadas en futuros intentos.

En resumen, registrar estadísticas detalladas y generar *logs* ofrece grandes beneficios a la hora de asegurar la eficiencia y calidad del sistema. Esta práctica está alineada con recomendaciones recientes en estudios sobre generación automática de código y pruebas unitarias, las cuales destacan la importancia de realizar un monitoreo continuo y detallado para lograr resultados fiables y escalables [17, 15, 1].

4.6. Selección del proyecto de pruebas: JInstagram

El proyecto seleccionado para evaluar nuestro sistema de generación automática de pruebas unitarias ha sido **JInstagram** [4]. Esta elección se basa en estudios previos que han identificado proyectos Java adecuados para probar técnicas automáticas de generación de tests. En concreto, Yuan et al. en [7] proponen una serie de criterios de selección claros, considerando factores como la calidad del código, la complejidad técnica, la capacidad de compilación y el uso de tecnologías comunes en la industria (por ejemplo, integración con APIs externas y uso de herramientas como Maven).

4.6.1. Motivos para elegir JInstagram

JInstagram destacó en estos estudios previos por las siguientes razones:

- **Complejidad y relevancia:** JInstagram funciona como un *wrapper* para la API de Instagram, lo que implica interactuar con servicios externos y realizar procesamiento de datos en tiempo real. Esto hace que el proyecto sea más complejo y represente un desafío real para evaluar la capacidad del modelo de lenguaje para generar pruebas efectivas en situaciones realistas, especialmente por la existencia de dependencias externas.
- **Construcción con Maven:** El proyecto utiliza Maven para compilar y gestionar sus dependencias, lo cual facilita la extracción automática de información

CAPÍTULO 4. DETALLES DE LA PROPUESTA

clave del archivo `pom.xml`, como versiones de librerías (por ejemplo, JUnit). Esto permite que el modelo genere pruebas más compatibles y compilables.

- **Mantenimiento y tamaño adecuado:** Con 6,303 líneas de código, JInstagram tiene un tamaño moderado, suficientemente grande para probar la generación automática de pruebas, pero sin ser excesivamente complejo. Además, usarse en contextos reales, garantiza que los resultados sean aplicables y válidos para estos entornos.

Siguiendo estos criterios definidos por Yuan et al. [7], JInstagram cumple todos los requisitos técnicos y de calidad necesarios para poder hacer una evaluación completa y detallada. Esta elección también refleja los desafíos reales que suelen encontrarse a la hora de generar tests en aplicaciones comerciales.

En resumen, **JInstagram** representa un caso de estudio ideal para validar la efectividad de nuestra solución, siguiendo criterios recomendados en estudios recientes [7].

4.7. Propuesta de experimentación

El objetivo de este experimento es evaluar cómo afecta la cantidad de contexto proporcionado al LLM en la calidad de las pruebas unitarias generadas automáticamente. Para ello, se propone reducir progresivamente la cantidad del contexto que se le da, conservando siempre la cantidad original de índices recuperados, pero truncando el contenido total. Debido a que los índices recuperados varían mucho en longitud (algunos pueden tener 40,000 caracteres y otros solo 2,000). En lugar de reducir la cantidad de índices recuperados, se truncará el contenido total del contexto para conservar el 100 %, 60 %, 40 % o 10 % del mismo. Dado que el contexto se recupera ordenado de mayor a menor relevancia, esta estrategia permite analizar el efecto de eliminar progresivamente la información menos relevante sobre la calidad

de las pruebas generadas.

4.7.1. Motivación y enfoque experimental

Proporcionar al modelo un contexto adecuado es fundamental para generar pruebas unitarias precisas y útiles. Estudios recientes muestran que incluir información recuperada mediante técnicas como RAG mejora considerablemente la calidad del código generado [17, 15]. Sin embargo, entregar demasiado contexto podría saturar la capacidad del modelo (ya que el número de tokens que puede procesar es limitado), aumentar costes innecesariamente (ya que se factura por tokens consumidos) o introducir datos que no aportan valor. Por otro lado, un contexto demasiado limitado podría dejar fuera información clave para la generación correcta de pruebas.

Por lo tanto, es esencial estudiar cómo la cantidad de contexto proporcionado influye realmente en la calidad de las pruebas generadas. Para ello, se plantea recortar progresivamente la información según los siguientes niveles:

- **100 %:** Se usa el contexto recuperado completo, sin eliminar nada.
- **60 %:** Se conserva el 60 % más relevante, descartando el resto.
- **40 %:** Se mantiene solamente el 40 % más relevante del contexto.
- **10 %:** Se incluye solo el 10 % más importante del contexto recuperado.

Este método permite controlar con precisión la cantidad total de información proporcionada, sin afectar la variedad de índices recuperados originalmente, dado que la longitud de estos puede variar significativamente.

4.7.2. Metodología de evaluación

Con cada configuración experimental (100 %, 60 %, 40 % y 10 %) se seguirá el proceso completo de generación y evaluación de pruebas generadas, que incluye los siguientes pasos:

CAPÍTULO 4. DETALLES DE LA PROPUESTA

1. **Generación del contexto:** Se recupera el contexto mediante RAG, ordenado desde lo más relevante hasta lo menos relevante. Luego, se trunca según el porcentaje definido para cada experimento.
2. **Generación y validación de pruebas:** Utilizando la API de Codestral, se generan las pruebas. Luego, se compilan y ejecutan para verificar si son correctas.
3. **Registro de métricas:** Se guardarán las estadísticas detalladas que se hayan generado sobre cada configuración, como la cantidad de *tokens* usados, la tasa de compilación, la tasa de ejecución exitosa y los tiempos de respuesta. Estos resultados permitirán comparar claramente el efecto de truncar el contexto.

La comparación de estas configuraciones permitirá entender cómo influye la cantidad de contexto proporcionado en la calidad final de las pruebas generadas. Por ejemplo, es posible que un contexto demasiado reducido (como el 10 %) sea insuficiente, mientras que usar siempre el contexto completo (100 %) podría incluir datos innecesarios que no aporten valor al modelo.



CAPÍTULO 4. DETALLES DE LA PROPUESTA

5. Experimentación y resultados

En este capítulo se presentan y analizan en detalle los resultados obtenidos en los experimentos realizados para la generación de pruebas unitarias. Se han realizado los experimentos variando el porcentaje de contexto recuperado con el RAG (100 %, 60 %, 40 % y 10 %) detallados en el Capítulo 4, y se han registrado las métricas propuestas como la cobertura de sentencias y ramas, la estabilidad de los tests (número de tests ejecutados, fallos, errores y tests omitidos), el consumo de tokens, y los tiempos de ejecución en diferentes fases. Con estos datos, se comparan nuestros resultados con los obtenidos por ChatUniTest [7] para el proyecto JInstagram [4] junto con los resultados de las pruebas generadas por los propios desarrolladores para el proyecto, y se discute en profundidad cómo la reducción del contexto afecta la calidad de la generación de tests.

5.1. Resumen y análisis de resultados

A continuación se presentan las tablas que resumen y agrupan los resultados obtenidos para cada configuración experimental y su análisis.

5.1.1. Comparación con ChatUniTest

La Tabla 5.1 muestra una comparación directa entre el mejor resultado de nuestra propuesta (utilizando el 100 % del contexto), los resultados obtenidos por

ChatUniTest para el proyecto JInstagram y los resultados de las pruebas generadas por los desarrolladores del proyecto ¹. Se comparan las coberturas de sentencias y de ramas, junto con las estadísticas de: tests ejecutados, fallos, errores y tests omitidos.

Métrica	Humanos	ChatUniTest	RAG
Cobertura de sentencias (%)	47	41,8	60
Cobertura por ramas (%)	24	13,4	39
Tests run	485	NC	522
Failures	0	NC	24
Errors	3	NC	42
Skipped	19	NC	0

Tabla 5.1: Comparación de resultados de pruebas: humanas, ChatUniTest [7] y la propuesta (con 100 % de contexto) para JInstagram [4]

Análisis

El análisis de la Tabla 5.1 muestra que, utilizando el 100 % del contexto, nuestro sistema logra una cobertura de sentencias significativamente mayor (60 %) y una cobertura por ramas de 39 %, lo que supera ampliamente los resultados obtenidos por ChatUniTest (41,8 % y 13,4 %, respectivamente) e incluso los resultados obtenidos por las pruebas generadas por humanos para el proyecto (47 % y 24 %, respectivamente).

Esto indica que el uso de un contexto completo permite generar pruebas que cubren un mayor número de escenarios y caminos de ejecución. Sin embargo, estos beneficios en términos de cobertura también se acompañan de ciertos inconvenientes en la estabilidad de la generación automática. Mientras que las pruebas humanas tienen muy pocos errores (0 fallos y 3 errores), nuestra propuesta tiene 24 fallos y 42 errores. Estos resultados pueden deberse tanto a que el modelo ha logrado generar

¹Los datos relativos a Tests run, Failures, Errors y Skipped para ChatUniTest no se encuentran disponibles en la literatura, por lo que se indica “NC” (No Conocido).

tests con una gran capacidad de detección como a la presencia de *asserts* erróneos. Es importante destacar que, aunque ChatUniTest no proporciona datos completos sobre las estadísticas de ejecución (se marca como NC), la comparación entre nuestras pruebas y las humanas resalta la necesidad de optimizar los mecanismos de corrección y contar con LLMs con mayores capacidades que puedan generar mejores pruebas con menos errores.

5.1.2. Comparación de cobertura según contexto

La Tabla 5.2 resume la evolución de la cobertura de sentencias y de ramas a medida que se reduce el porcentaje del contexto.

Métrica	100 %	60 %	40 %	10 %
Cobertura de sentencias (%)	60	48	28	10
Cobertura por ramas (%)	39	22	12	1

Tabla 5.2: Cobertura de pruebas según el porcentaje de contexto recuperado

Análisis

Los resultados en la Tabla 5.2 indican que a medida que se reduce el contexto, la cobertura disminuye de forma progresiva. Con el 100 % del contexto se obtienen mejores resultados (60 % y 39 %), mientras que con solo el 10 % la cobertura se desploma al 10 % y 1 %, respectivamente.

Este comportamiento muestra que la información de las secciones finales del contexto (los índices menos relevantes), aunque se considera de menor relevancia, aporta datos críticos para la generación de tests que permiten alcanzar una mayor cobertura. Además, también nos muestra que el LLM sin contexto genera unos resultados pésimos.

5.1.3. Resumen detallado de las métricas analizadas

La Tabla 5.3 agrupa los principales resultados extraídos del análisis de los CSV. Se resumen el número total de llamadas al LLM, el porcentaje de clases de tests compilables, y los promedios de caracteres y tokens, tanto a nivel individual como global.

Parámetro	100 %	60 %	40 %	10 %
Nº total de llamadas	123	129	177	208
Total de clases (sin tests)	86	86	86	86
Clases de tests compilables	74	67	42	26
Clases no compilables (%)	13.95 %	22.09 %	51.16 %	69.77 %
Compilan en 1º intento (%)	74.42 %	73.26 %	45.35 %	29.07 %
Compilan en 2º intento (%)	8.14 %	3.49 %	3.49 %	0.00 %
Compilan en 3 ^{er} intento (%)	3.49 %	1.16 %	0.00 %	1.16 %
Promedio de caracteres por clase	19731	12328	4551	2518
Promedio de tokens por clase	5403	3687	1767	1445
Promedio de caracteres global	21483	13987	4545	2546
Promedio de tokens global	5835	4227	1801	1538
Promedio de <code>completion_tokens</code>	695	860	676	906
Total de tokens consumidos	717646	545315	318833	319949

Tabla 5.3: Resumen del informe del CSV y consumo de tokens

Análisis

El análisis de la Tabla 5.3 permite también extraer diversas conclusiones que tienen implicaciones importantes para la generación automática de pruebas unitarias:

Se observa que al disminuir el porcentaje de contexto (pasando de 100 % a 10 %), el promedio de caracteres por clase se reduce drásticamente, de aproximadamente

CAPÍTULO 5. EXPERIMENTACIÓN Y RESULTADOS

19,730 caracteres a tan solo 2,517 caracteres. De manera similar, el promedio de tokens por clase desciende de 5,402 a 1,444 tokens.

No obstante, esta reducción se acompaña de un deterioro progresivo en la estabilidad y de la generación, ya que aumenta el porcentaje de clases de tests que no compilan, del 13.95 % en la configuración de 100 % de contexto a 69.77 % en la configuración de 10 %. Además, la proporción de tests que compilan en el primer intento se reduce de 74.42 % a 29.07 %. Estos datos nos muestran que un contexto más extenso es fundamental para la compilabilidad en el proceso de generación; la información adicional contenida en el contexto completo es crucial para generar tests con una buena sintaxis y, por lo tanto, compilables.

A pesar de que la reducción del contexto lleva a una disminución en el número total de tokens consumidos durante la generación de las pruebas (de 717,646 tokens en la configuración de 100 % a 319,949 tokens en la de 10 %), este ahorro viene acompañado de una degradación enorme en la calidad de los tests generados. Esto sugiere que, aunque se optimiza el coste en términos de tokens, la reducción de contexto perjudica la capacidad del LLM para generar resultados robustos y coherentes.

Los datos muestran que mantener un *prompt* extenso (100 % del contexto) proporciona una mayor cantidad de información, lo que se traduce en una mejor comprensión del entorno de la clase a testear. Esto se refleja en una mayor cobertura y en un menor porcentaje de tests no compilables. La información adicional, aunque incrementa el consumo de tokens, es esencial para que el LLM genere pruebas unitarias que logren cubrir adecuadamente las diferentes ramas de ejecución. Por el contrario, cuando el contexto se reduce, la escasez de información resulta en tests con baja cobertura y alta tasa de errores de compilación, lo cual respalda la hipótesis de que la inclusión de un contexto amplio es indispensable para la generación de pruebas de calidad.

En conclusión, el análisis del informe del CSV y del consumo de tokens resalta la importancia de proporcionar un contexto amplio al LLM. Aunque la reducción del contexto puede disminuir el coste en términos de tokens, ello se traduce en una

notable degradación en la estabilidad y calidad de los tests generados, confirmando así la necesidad de utilizar el 100 % del contexto para obtener mejores resultados.

5.1.4. Análisis de la ejecución de los tests según el porcentaje de contexto

La Tabla 5.4 resume los resultados obtenidos en la ejecución de los tests para cada configuración de contexto. Se observa que, utilizando el 100 % del contexto, se ejecutaron 522 tests, de los cuales se registraron 24 fallos y 42 errores, sin tests omitidos. Con una reducción al 60 %, el número de tests ejecutados disminuye a 430, y los fallos y errores se reducen a 21 cada uno. Al reducir el contexto al 40 %, se ejecutaron 309 tests con 7 fallos y 2 errores, mientras que con solo el 10 % de contexto se obtuvieron 140 tests sin registrar ni fallos ni errores.

Métrica	100 %	60 %	40 %	10 %
Tests run	522	430	309	140
Failures	24	21	7	0
Errors	42	21	2	0
Skipped	0	0	0	0

Tabla 5.4: Resultados de ejecución de tests según porcentaje de contexto

Análisis

El análisis de la Tabla 5.4 nos muestra que el uso del 100 % del contexto proporciona una mayor cantidad de tests generados, lo que se traduce en una mayor cobertura potencial del código. Sin embargo, esta configuración también presenta un mayor número de fallos y errores, lo que indica que la inclusión de toda la información, aunque es valiosa, puede introducir complejidad adicional y aumentar la probabilidad de errores durante la ejecución de los tests. Como se ha comentado anteriormente,

CAPÍTULO 5. EXPERIMENTACIÓN Y RESULTADOS

estos errores pueden deberse tanto a que el modelo ha logrado generar tests con una gran capacidad de detección como a la presencia de *asserts* erróneos. Al reducir el contexto al 60 % y 40 %, se observa una disminución gradual tanto en el número de tests ejecutados como en los errores, lo que sugiere que eliminar parte de la información menos relevante mejora la estabilidad de los tests generados, aunque a costa de una cobertura potencialmente menor y más superficial.

En la configuración del 10 % de contexto, los resultados muestran que, aunque se obtiene una ejecución sin fallos ni errores, el número de tests generados se reduce enormemente (a 140), lo que nos lleva a una cobertura muy reducida. Esto refuerza la idea de que un contexto excesivamente reducido impide al LLM capturar información crítica necesaria para generar tests robustos y variados. En conclusión, existe un claro compromiso entre la cantidad de contexto y la calidad de los tests generados: mientras más completo es el contexto, mayor es la cobertura y diversidad de tests, pero también aumenta la complejidad y la tasa de errores. Por el contrario, un contexto muy reducido mejora la estabilidad pero limita significativamente la cobertura y calidad de los tests.

5.1.5. Resumen de tiempos de generación de tests

La Tabla 5.5 agrupa los tiempos de generación de tests tanto en promedio por clase como a nivel global para cada configuración.

Configuración	100 %	60 %	40 %	10 %
Tiempo promedio por clase (s)	4.25	4.27	3.85	4.93
Tiempo global (s)	4.58	5.47	4.19	5.52

Tabla 5.5: Resumen de tiempos de generación de tests por porcentaje de contexto

Análisis

Los tiempos de generación por clase son relativamente similares entre las configuraciones, con ligeras variaciones. Sin embargo, se observa que la configuración con el 10 % de contexto presenta un aumento en el tiempo global por clase (5.52 s), lo que puede atribuirse a la mayor cantidad de iteraciones necesarias para compensar los errores de compilación por la pérdida de información crítica. Esto sugiere que, aunque un contexto reducido puede acelerar ciertos procesos (por ejemplo, la consulta del índice), la necesidad de regenerar pruebas para corregir errores de compilación aumenta el tiempo total.

5.1.6. Resumen de tiempos totales del proceso

La Tabla 5.6 agrupa los tiempos totales del proceso, abarcando desde la creación del RAG y del índice FAISS hasta la ejecución global del script.

Fase	100 %	60 %	40 %	10 %
RAG e índice FAISS (s)	64.24	58.42	51.09	62.31
Generación de tests (s)	950.65	1097.40	1259.61	1750.65
Ejecución de tests con Maven (s)	9.20	4.25	3.84	3.80
Generación del informe (s)	1.7e-8	1.87e-7	1e-9	0.0045
Ejecución total del script (s)	1024.10	1160.07	1314.54	1816.77

Tabla 5.6: Resumen de tiempos totales del proceso por porcentaje de contexto

Análisis

La Tabla 5.6 muestra que la generación de tests se ve afectada significativamente al reducir el contexto. Con el 100 % del contexto, la generación de tests tarda 950.65 s, mientras que con el 10 % se incrementa a 1750.65 s, casi el doble. Esto se debe a la necesidad de realizar múltiples iteraciones para tratar de corregir errores producidos por tener un contexto enormemente limitado. Por otro lado, la ejecución de tests con

Maven es más rápida con menos contexto, debido a que el número de tests generados válidos es menor, pero esto se traduce en una cobertura muy reducida.

En conjunto, los tiempos totales del proceso aumentan notablemente al reducir el contexto, lo que refuerza la conclusión de que un contexto amplio no solo mejora la calidad de la generación, la compilabilidad y la cobertura, sino también la eficiencia global y el consumo adicional de tokens durante la regeneración al evitar múltiples iteraciones de esta.

5.2. Discusión final del análisis

Los resultados de los experimentos muestran de forma concluyente que:

- Con el 100 % del contexto se obtienen los mejores resultados en términos de cobertura de sentencias y ramas, superando a las configuraciones de 60 %, 40 % y 10 %. La comparación con ChatUniTest y los tests generados por los desarrolladores del proyecto, confirma que nuestro enfoque, al conservar todo el contexto que obtenemos del RAG, permite generar pruebas más completas, aunque con un mayor número de errores ya que son más complejas y completas.
- La reducción del contexto disminuye el consumo total de tokens, lo cual puede ser importante desde el punto de vista de la reducción costes. Sin embargo, este ahorro se ve contrarrestado por una mayor inestabilidad en la generación, evidenciada en el incremento del porcentaje de clases de tests que no compilan, lo que afecta negativamente a la cobertura final y al tiempo de generación. Además al realizarse intentos de generación adicionales, también se consumen tokens extra.
- Los tiempos de generación y los tiempos totales del script aumentan drásticamente en configuraciones con menor contexto, debido a la necesidad de múltiples iteraciones de corrección. Esto sugiere que invertir en un contexto

CAPÍTULO 5. EXPERIMENTACIÓN Y RESULTADOS

más completo resulta en una eficiencia y calidad global superior, ya que se reducen los ciclos de retroalimentación necesarios.

En conclusión, la experimentación respalda la hipótesis de que mantener un contexto amplio (el 100 % de lo que obtenemos del RAG) es fundamental para obtener pruebas unitarias de alta calidad, tanto en términos de cobertura como de estabilidad, pese al mayor consumo de tokens. Además, utilizar la técnica propuesta ha demostrado mejores resultados que las pruebas generadas por los propios desarrolladores del proyecto y la bibliografía existente.

6. Conclusiones y trabajo futuro

En este capítulo se detallan las conclusiones obtenidas tras analizar los resultados obtenidos en los experimentos realizados para la generación de pruebas unitarias. Finalmente, se detallan las principales líneas de investigación futura.

6.1. Conclusiones

El trabajo demuestra que enriquecer el contexto proporcionado a los modelos de lenguaje para la generación automática de pruebas unitarias mejora de forma significativa la calidad y cobertura de las pruebas generadas. En concreto, se ha comprobado que:

La utilización de un contexto completo, es decir, el 100 % del generado por el RAG, permite alcanzar coberturas de sentencias y ramas notablemente superiores (60 % y 39 %, respectivamente), superando tanto los resultados obtenidos por una de las mejores herramientas existentes del estado del arte, como los de los tests generados manualmente por desarrolladores. Esto indica que disponer de mayor información del proyecto favorece la comprensión profunda del código y, por ende, la generación de tests más robustos.

La incorporación de técnicas basadas en Generación Aumentada por Recuperación (RAG) ha demostrado ser efectiva para mitigar la limitación inherente a la cantidad de tokens que pueden procesarse en cada llamada de los LLMs. Además, al integrar información extraída de diferentes fuentes (contexto del código generado por el RAG

CAPÍTULO 6. CONCLUSIONES Y TRABAJO FUTURO

y datos del archivo pom.xml) en el *prompt*, se reducen errores de compilación y se generan pruebas unitarias con *assertions* más precisas.

Se observa una clara degradación en la calidad de las pruebas cuando se reduce el porcentaje de contexto (la cobertura se reduce drásticamente, y aumenta el porcentaje de tests que no compilan). Además, los tiempos de generación aumentan enormemente, debido a los intentos de regeneración, lo que a su vez consume también más tokens. Esto refuerza la importancia de disponer de un contexto amplio para mantener la estabilidad, calidad y robustez en la generación automática de las pruebas.

Sin embargo, el análisis comparativo también revela que el incremento en la cobertura viene acompañado de un mayor número de fallos y errores en la ejecución de los tests, lo que puede deberse tanto a que el modelo ha logrado generar tests con una gran capacidad de detección como a la presencia de *asserts* erróneos. La complejidad del *prompt* al utilizar un contexto más completo puede afectar la estabilidad de los tests generados, evidenciado en fallos y errores en comparación con los errores casi inexistentes presentes en las pruebas humanas. Estos resultados indican que, si bien un contexto extenso mejora la cobertura, es crucial contar con LLMs con grandes capacidades que puedan generar mejores pruebas con menos errores y mejorar, por tanto, la robustez de los tests generados.

6.2. Trabajo futuro

Las líneas de investigación futura se centrarán en la optimización de los mecanismos de corrección iterativa y en la mejora de la estabilidad de la generación de pruebas. Una idea es la incorporación de técnicas avanzadas de validación y post-procesamiento automático, que permitan reducir los errores de compilación sin necesidad de más iteraciones. Por otro lado, se explorará el desarrollo de un sistema de corrección en tiempo real que ajuste dinámicamente el contexto suministrado en función del desempeño obtenido en pruebas previas, con el fin de mejorar la eficiencia

CAPÍTULO 6. CONCLUSIONES Y TRABAJO FUTURO

global del proceso.

Otra línea de trabajo futuro es la experimentación con diferentes modelos de lenguaje y estrategias de *prompting*. Aunque en esta propuesta se ha empleado la API Mistral con el modelo Codestral 25.01 [3] junto con `microsoft/graphcodebert-base` [1] para la generación de embeddings y la recuperación del contexto, futuros estudios podrían evaluar la integración de otros LLMs y modelos de embeddings alternativos y con mayores capacidades. Esto permitirá determinar cuál ofrece la mejor relación entre cobertura, estabilidad y coste, extendiendo la aplicabilidad del enfoque a otros lenguajes de programación y contextos de desarrollo.



CAPÍTULO 6. CONCLUSIONES Y TRABAJO FUTURO

Anexo A. Manual de código, despliegue y uso

Este manual de código ha sido creado con el objetivo de explicar la estructuración del código fuente del sistema, así como proporcionar una guía detallada para el mantenimiento, configuración y despliegue del proyecto. Está dirigido a desarrolladores y técnicos que necesiten actualizar, modificar o comprender el funcionamiento del sistema.

A.1. Código fuente y organización del proyecto

El código fuente desarrollado junto con los resultados de la experimentación se encuentra disponible en el siguiente repositorio [33].

Para clonar el repositorio, utilice:

```
1 git clone https://github.com/AlfonsoCF02/TFM_RAG-test-generation.git
```

A.2. Prerrequisitos

Antes de desplegar y ejecutar el proyecto, asegúrese de contar con los siguientes elementos:

ANEXO A. MANUAL DE CÓDIGO, DESPLIEGUE Y USO

- **Python:** Versión 3.9.13 o superior
- **JDK:** Se recomienda tener instalado JDK 11 o superior.
- **Maven:** Versión 3.6.0 o superior.

A.2.1. Dependencias Python

El script utiliza módulos de la biblioteca estándar y varias dependencias externas. A continuación se detalla cada uno:

Módulos de la Biblioteca Estándar

- `os`
- `re`
- `sys`
- `glob`
- `subprocess`
- `xml.etree.ElementTree`
- `csv`
- `time`
- `datetime`

Dependencias externas

- `numpy`: Para operaciones numéricas.
- `faiss`: (o `faiss-cpu` en entornos CPU) para búsquedas vectoriales.

- `torch`: Para manejo de tensores y modelos.
- `requests`: Para realizar peticiones HTTP.
- `transformers`: Para utilizar `AutoTokenizer` y `AutoModel`.
- `python-dotenv`: Para gestionar variables de entorno.

Para instalar estas dependencias, se recomienda ejecutar el siguiente comando:

```
1 pip install [Nombre de la dependencia]
```

A.2.2. Archivo `requirements.txt`

El archivo `requirements.txt` es una gran ayuda para la instalación de las dependencias necesarias para ejecutar el script ya que permite instalarlas automáticamente:

El archivo `requirements.txt` contiene:

```
1 numpy
2 faiss-cpu
3 torch
4 requests
5 transformers
6 python-dotenv
```

Listado A.1: Archivo `requirements.txt`

Para instalarlo se debe ejecutar:

```
1 pip install -r requirements.txt
```

A.2.3. Creación y configuración del archivo `.env`

En la raíz del proyecto debe ubicarse un archivo `.env` que contenga, las siguientes variables:

```
1 API_KEY=[su_api_key]
2 BASE_SOURCE_DIR=[su_ruta_al_proyecto]
```

Reemplace *[su_api_key]* por la API key proporcionada por el servicio de Mistral y *[su_ruta_al_proyecto]* por la ruta a la carpeta que contiene el código fuente Java.

El script depende de dichas variables para poder ejecutarse. La variable `BASE_SOURCE_DIR` le permite localizar los archivos Java del proyecto Maven. Es fundamental que se configure correctamente la ruta a la carpeta que contiene el código fuente Java (`\src\main\java`). Por ejemplo:

```
1 BASE_SOURCE_DIR=C:\Users\alfon\Documents\GitHub
2 \TFM_RAG-test-generation\jInstagram-master\src\main\java
```

Listado A.2: Configuración de la ruta a los archivos Java

A.3. Configuración del `pom.xml` para Maven

Para compilar correctamente el proyecto y generar los análisis de cobertura de las pruebas unitarias, es necesario incluir en el archivo `pom.xml` los siguientes plugins:

```
1 <!-- Plugin para la ejecución de tests -->
2 <plugin>
3   <groupId>org.apache.maven.plugins</groupId>
4   <artifactId>maven-surefire-plugin</artifactId>
5   <version>2.22.2</version>
6   <configuration>
7     <testFailureIgnore>true</testFailureIgnore>
8   </configuration>
9 </plugin>
10 <!-- Plugin JaCoCo para medir la cobertura -->
```

ANEXO A. MANUAL DE CÓDIGO, DESPLIEGUE Y USO

```
11 <plugin>
12   <groupId>org.jacoco</groupId>
13   <artifactId>jacoco-maven-plugin</artifactId>
14   <version>0.8.8</version>
15   <executions>
16     <!-- Prepara el agente JaCoCo para instrumentar el código -->
17     <execution>
18       <id>prepare-agent</id>
19       <goals>
20         <goal>prepare-agent</goal>
21       </goals>
22     </execution>
23     <!-- Genera el reporte de cobertura después de ejecutar los
24           tests -->
25     <execution>
26       <id>report</id>
27       <phase>test</phase>
28       <goals>
29         <goal>report</goal>
30       </goals>
31       <configuration>
32         <verbose>true</verbose>
33       </configuration>
34     </execution>
35   </executions>
36 </plugin>
```

Listado A.3: Configuración Maven para tests y cobertura con JaCoCo

Esta configuración garantiza que:

- Los tests se ejecuten correctamente, ignorando fallos para no detener la compilación.
- Se prepare JaCoCo para analizar la cobertura del código.
- Se genere un reporte de cobertura de código en la fase de test, mostrando un resumen detallado en la consola.

A.4. Despliegue y uso

A.4.1. Ejecución del script de generación de pruebas

Para ejecutar el script que genera las pruebas unitarias:

1. Asegúrese de que el archivo `.env` y la ruta al proyecto están configurados correctamente.
2. Instale las dependencias de Python:

```
1 pip install -r requirements.txt
```

3. Ejecute el script:

```
1 python SCRIPT_RAG-test-generation.py
```

Una vez ejecutado el script y generadas las pruebas unitarias, podrá ver los resultados en consola y en los informes generados.

Bibliografía

- [1] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang y Ming Zhou. *GraphCodeBERT: Pre-training Code Representations with Data Flow*. 2021. arXiv: 2009.08366 [cs.SE]. URL: <https://arxiv.org/abs/2009.08366>.
- [2] Jaroslav Johnson, Matthijs Douze y Hervé Jégou. «FAISS: A library for efficient similarity search and clustering of dense vectors». En: *Proceedings of the 30th Conference on Neural Information Processing Systems (NIPS 2017)*. 2017. URL: <https://github.com/facebookresearch/faiss>.
- [3] Mistral AI Team. *Codestral 25.01 - A big upgrade to Mistral's coding model*. Mistral AI News (Jan 13, 2025). 2025. URL: <https://mistral.ai/news/codestral-2501>.
- [4] Sachin Handiekar. *jInstagram*. <https://github.com/sachin-handiekar/jInstagram>. 2015.
- [5] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng y Jianwei Yin. *ChatUniTest: A Framework for LLM-Based Test Generation*. 2024. arXiv: 2305.04764 [cs.SE]. URL: <https://arxiv.org/abs/2305.04764>.
- [6] Wendkûuni C. Ouédraogo, Kader Kaboré, Haoye Tian, Yewei Song, Anil Koyuncu, Jacques Klein, David Lo y Tegawendé F. Bissyandé. *Large-scale,*

- Independent and Comprehensive study of the power of LLMs for test case generation*. 2024. arXiv: 2407.00225 [cs.SE]. URL: <https://arxiv.org/abs/2407.00225>.
- [7] Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng y Yiling Lou. «Evaluating and Improving ChatGPT for Unit Test Generation». En: *Proc. ACM Softw. Eng.* 1.FSE (jul. de 2024). DOI: 10.1145/3660783. URL: <https://doi.org/10.1145/3660783>.
- [8] Jiho Shin, Reem Aleithan, Hadi Hemmati y Song Wang. *Retrieval-Augmented Test Generation: How Far Are We?* 2024. arXiv: 2409.12682 [cs.SE]. URL: <https://arxiv.org/abs/2409.12682>.
- [9] Lin Yang, Chen Yang, Shutao Gao, Weijing Wang, Bo Wang, Qihao Zhu, Xiao Chu, Jianyi Zhou, Guangtai Liang, Qianxiang Wang y Junjie Chen. *On the Evaluation of Large Language Models in Unit Test Generation*. 2024. arXiv: 2406.18181 [cs.SE]. URL: <https://arxiv.org/abs/2406.18181>.
- [10] Siqi Gu, Qunjun Zhang, Chunrong Fang, Fangyuan Tian, Liuchuan Zhu, Jianyi Zhou y Zhenyu Chen. *TestART: Improving LLM-based Unit Testing via Co-evolution of Automated Generation and Repair Iteration*. 2024. arXiv: 2408.03095 [cs.SE]. URL: <https://arxiv.org/abs/2408.03095>.
- [11] Gordon Fraser y Andrea Arcuri. «EvoSuite: automatic test suite generation for object-oriented software». En: *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering*. ESEC/FSE '11. Szeged, Hungary: Association for Computing Machinery, 2011, págs. 416-419. ISBN: 9781450304436. DOI: 10.1145/2025113.2025179. URL: <https://doi.org/10.1145/2025113.2025179>.
- [12] C. Pacheco y M.D. Ernst. *Randoop: A Tool for Automatic Unit Test Generation*. 2007. URL: <https://homes.cs.washington.edu/~mernst/pubs/pacheco-randoop-oopsla2007.pdf>.

BIBLIOGRAFÍA

- [13] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, Shao Kun Deng y Neel Sundaresan. *Unit Test Case Generation with Transformers and Focal Context*. 2021. arXiv: 2009.05617 [cs.SE]. URL: <https://arxiv.org/abs/2009.05617>.
- [14] Saranya Alagarsamy, Chakkrit Tantithamthavorn y Aldeida Aleti. *A3Test: Assertion-Augmented Automated Test Case Generation*. 2023. arXiv: 2302.10352 [cs.SE]. URL: <https://arxiv.org/abs/2302.10352>.
- [15] Max Schäfer, Sarah Nadi, Aryaz Eghbali y Frank Tip. *An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation*. 2023. arXiv: 2302.06527 [cs.SE]. URL: <https://arxiv.org/abs/2302.06527>.
- [16] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri y Siddhartha Sen. «CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models». En: *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 2023, págs. 919-931. DOI: 10.1109/ICSE48619.2023.00085.
- [17] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel y Douwe Kiela. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. 2021. arXiv: 2005.11401 [cs.CL]. URL: <https://arxiv.org/abs/2005.11401>.
- [18] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen y Wen-tau Yih. *Dense Passage Retrieval for Open-Domain Question Answering*. 2020. arXiv: 2004.04906 [cs.CL]. URL: <https://arxiv.org/abs/2004.04906>.
- [19] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat y Ming-Wei Chang. *REALM: Retrieval-Augmented Language Model Pre-Training*. 2020. arXiv: 2002.08909 [cs.CL]. URL: <https://arxiv.org/abs/2002.08909>.

- [20] Rahul Gupta, Soham Pal, Aditya Kanade y Shirish Shevade. «DeepFix: Fixing Common C Language Errors by Deep Learning». En: *Proceedings of the AAAI Conference on Artificial Intelligence* 31.1 (feb. de 2017). DOI: 10.1609/aaai.v31i1.10742. URL: <https://ojs.aaai.org/index.php/AAAI/article/view/10742>.
- [21] Xin Yin, Chao Ni, Xinrui Li, Liushan Chen, Guojun Ma y Xiaohu Yang. *Enhancing LLM's Ability to Generate More Repository-Aware Unit Tests Through Precise Contextual Information Injection*. 2025. arXiv: 2501.07425 [cs.SE]. URL: <https://arxiv.org/abs/2501.07425>.
- [22] Chao Ni, Xiaoya Wang, Liushan Chen, Dehai Zhao, Zhengong Cai, Shaohua Wang y Xiaohu Yang. *CasModaTest: A Cascaded and Model-agnostic Self-directed Framework for Unit Test Generation*. 2024. arXiv: 2406.15743 [cs.SE]. URL: <https://arxiv.org/abs/2406.15743>.
- [23] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel y Douwe Kiela. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. 2021. arXiv: 2005.11401 [cs.CL]. URL: <https://arxiv.org/abs/2005.11401>.
- [24] Md Rizwan Parvez, Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray y Kai-Wei Chang. *Retrieval Augmented Code Generation and Summarization*. 2021. arXiv: 2108.11601 [cs.SE]. URL: <https://arxiv.org/abs/2108.11601>.
- [25] Tomas Mikolov, Kai Chen, Greg Corrado y Jeffrey Dean. *Efficient Estimation of Word Representations in Vector Space*. 2013. arXiv: 1301.3781 [cs.CL]. URL: <https://arxiv.org/abs/1301.3781>.

- [26] Uri Alon, Meital Zilberstein, Omer Levy y Eran Yahav. *code2vec: Learning Distributed Representations of Code*. 2018. arXiv: 1803.09473 [cs.LG]. URL: <https://arxiv.org/abs/1803.09473>.
- [27] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang y Ming Zhou. «CodeBERT: A Pre-Trained Model for Programming and Natural Languages». En: *Findings of the Association for Computational Linguistics: EMNLP 2020*. Ed. por Trevor Cohn, Yulan He y Yang Liu. Online: Association for Computational Linguistics, nov. de 2020, págs. 1536-1547. DOI: 10.18653/v1/2020.findings-emnlp.139. URL: <https://aclanthology.org/2020.findings-emnlp.139/>.
- [28] Jeff Johnson, Matthijs Douze y Hervé Jégou. *Billion-scale similarity search with GPUs*. 2017. arXiv: 1702.08734 [cs.CV]. URL: <https://arxiv.org/abs/1702.08734>.
- [29] Erik Bernhardsson. *Annoy: Approximate Nearest Neighbors Oh Yeah*. <https://github.com/spotify/annoy>. 2025.
- [30] Owen Pendrigh Elliott y Jesse Clark. *The Impacts of Data, Ordering, and Intrinsic Dimensionality on Recall in Hierarchical Navigable Small Worlds*. 2024. arXiv: 2405.17813 [cs.IR]. URL: <https://arxiv.org/abs/2405.17813>.
- [31] Charu C. Aggarwal, Alexander Hinneburg y Daniel A. Keim. «On the Surprising Behavior of Distance Metrics in High Dimensional Spaces». En: *Proceedings of the 8th International Conference on Database Theory*. ICDT '01. Berlin, Heidelberg: Springer-Verlag, 2001, págs. 420-434. ISBN: 3540414568.
- [32] Mayur Datar, Nicole Immorlica, Piotr Indyk y Vahab S. Mirrokni. «Locality-sensitive hashing scheme based on p-stable distributions». En: *Proceedings of the Twentieth Annual Symposium on Computational Geometry*. SCG '04. Brooklyn, New York, USA: Association for Computing Machinery, 2004, págs. 253-262.

BIBLIOGRAFÍA

ISBN: 1581138857. DOI: 10.1145/997817.997857. URL: <https://doi.org/10.1145/997817.997857>.

- [33] Alfonso Cabezas Fernández. *Repositorio con los resultados y el código fuente de la propuesta*. https://github.com/AlfonsoCF02/TFM_RAG-test-generation.git. 2025.